

Simulation Battle of Grunwald

Generated by Doxygen 1.16.1

1 Simulation Battle of Grunwald	1
1.1 Main Modules	1
1.2 Generate Documentation	1
1.3 UML Diagrams	1
2 Simulation Battle of Grunwald	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Class Documentation	11
6.1 AlliedHelp Class Reference	11
6.1.1 Detailed Description	12
6.1.2 Member Function Documentation	12
6.1.2.1 apply()	12
6.1.2.2 getName()	13
6.2 Archer Class Reference	13
6.2.1 Detailed Description	15
6.2.2 Constructor & Destructor Documentation	15
6.2.2.1 Archer()	15
6.2.3 Member Function Documentation	15
6.2.3.1 attack_Enemy()	15
6.2.3.2 get_Type()	16
6.2.3.3 get_WeatherModifier()	16
6.2.3.4 take_Damage()	16
6.3 Army Class Reference	17
6.3.1 Detailed Description	18
6.3.2 Constructor & Destructor Documentation	18
6.3.2.1 Army()	18
6.3.3 Member Function Documentation	18
6.3.3.1 add_Reinforcements()	18
6.3.3.2 count_AliveArchers()	19
6.3.3.3 count_AliveCavalry()	19
6.3.3.4 count_AliveKnights()	19
6.3.3.5 count_AliveWarriors()	19
6.3.3.6 decrease_Morale()	19
6.3.3.7 get_ArmySize()	19
6.3.3.8 get_moraleLevel()	20

6.3.3.9	get_MoraleModifier()	20
6.3.3.10	get_Name()	20
6.3.3.11	get_RandomAliveWarrior()	20
6.3.3.12	get_Warriors()	20
6.3.3.13	increase_Morale()	21
6.4	Battle Class Reference	21
6.4.1	Detailed Description	22
6.4.2	Constructor & Destructor Documentation	22
6.4.2.1	Battle()	22
6.4.3	Member Function Documentation	23
6.4.3.1	attach()	23
6.4.3.2	do_Round()	23
6.4.3.3	finishBattle()	23
6.4.3.4	get_CurrentEvent()	23
6.4.3.5	get_HussarsArmy()	23
6.4.3.6	get_Rounds()	24
6.4.3.7	get_Stats()	24
6.4.3.8	get_TeutonicArmy()	24
6.4.3.9	get_Weather()	24
6.4.3.10	get_Winner()	24
6.4.3.11	is_finished()	24
6.4.3.12	notify()	25
6.4.3.13	start_Battle()	25
6.5	BattleObserver Class Reference	25
6.5.1	Detailed Description	27
6.5.2	Member Function Documentation	27
6.5.2.1	update()	27
6.6	BattlePanel Class Reference	27
6.6.1	Detailed Description	30
6.6.2	Constructor & Destructor Documentation	30
6.6.2.1	BattlePanel()	30
6.6.3	Member Function Documentation	30
6.6.3.1	setBattleNumber()	30
6.6.3.2	setCurrentEvent()	30
6.6.3.3	setHussarsMorale()	31
6.6.3.4	setHussarsProbability()	31
6.6.3.5	setRounds()	31
6.6.3.6	setTeutonicMorale()	31
6.6.3.7	setTeutonicProbability()	31
6.6.3.8	setWeather()	32
6.6.3.9	setWinner()	32
6.6.3.10	update()	32

6.7 BattleReport Class Reference	32
6.7.1 Detailed Description	33
6.7.2 Member Function Documentation	33
6.7.2.1 save()	33
6.7.2.2 save_skipped()	33
6.8 BattleStats Class Reference	34
6.8.1 Detailed Description	35
6.8.2 Member Function Documentation	35
6.8.2.1 add_HussarsDamage()	35
6.8.2.2 add_HussarsKill()	35
6.8.2.3 add_TeutonicDamage()	35
6.8.2.4 add_TeutonicKill()	35
6.8.2.5 get_HussarsDamage_Dealt()	35
6.8.2.6 get_HussarsKills()	36
6.8.2.7 get_TeutonicDamage_Dealt()	36
6.8.2.8 get_TeutonicKills()	36
6.9 Cavalryman Class Reference	36
6.9.1 Detailed Description	39
6.9.2 Constructor & Destructor Documentation	39
6.9.2.1 Cavalryman()	39
6.9.3 Member Function Documentation	39
6.9.3.1 attack_Energy()	39
6.9.3.2 get_Type()	40
6.9.3.3 get_WeatherModifier()	40
6.9.3.4 take_Damage()	40
6.10 CommanderDeath Class Reference	41
6.10.1 Detailed Description	42
6.10.2 Member Function Documentation	42
6.10.2.1 apply()	42
6.10.2.2 getName()	42
6.11 ConfigLoader Class Reference	43
6.11.1 Detailed Description	43
6.11.2 Member Function Documentation	43
6.11.2.1 LoadFromFile()	43
6.12 ControlPanel Class Reference	44
6.12.1 Detailed Description	45
6.12.2 Constructor & Destructor Documentation	45
6.12.2.1 ControlPanel()	45
6.12.3 Member Function Documentation	46
6.12.3.1 get_ExitButton()	46
6.12.3.2 get_LoadButton()	46
6.12.3.3 get_SpeedComboBox()	46

6.12.3.4 get_StartButton()	46
6.13 FractionConfig Struct Reference	47
6.13.1 Detailed Description	48
6.13.2 Member Data Documentation	48
6.13.2.1 archer_count	48
6.13.2.2 archer_stats	48
6.13.2.3 cavalry_count	48
6.13.2.4 cavalry_stats	49
6.13.2.5 fraction	49
6.13.2.6 knight_count	49
6.13.2.7 knight_stats	49
6.13.2.8 valid	49
6.14 HistoryPanel Class Reference	50
6.14.1 Detailed Description	51
6.14.2 Constructor & Destructor Documentation	51
6.14.2.1 HistoryPanel()	51
6.14.3 Member Function Documentation	51
6.14.3.1 addBattleResult()	51
6.14.3.2 clearHistoryList()	51
6.15 HussarsPanel Class Reference	52
6.15.1 Detailed Description	53
6.15.2 Constructor & Destructor Documentation	54
6.15.2.1 HussarsPanel()	54
6.15.3 Member Function Documentation	54
6.15.3.1 setAlive()	54
6.15.3.2 setArchers()	54
6.15.3.3 setCavalrymans()	54
6.15.3.4 setDamage()	54
6.15.3.5 setKills()	55
6.15.3.6 setKnights()	55
6.16 Knight Class Reference	55
6.16.1 Detailed Description	58
6.16.2 Constructor & Destructor Documentation	58
6.16.2.1 Knight()	58
6.16.3 Member Function Documentation	58
6.16.3.1 attack_Energy()	58
6.16.3.2 get_Type()	59
6.16.3.3 get_WeatherModifier()	59
6.16.3.4 take_Damage()	59
6.17 MainWindow Class Reference	60
6.17.1 Detailed Description	61
6.17.2 Constructor & Destructor Documentation	61

6.17.2.1 MainWindow()	61
6.18 PanicEvent Class Reference	61
6.18.1 Detailed Description	62
6.18.2 Member Function Documentation	62
6.18.2.1 apply()	62
6.18.2.2 getName()	63
6.19 Random Class Reference	63
6.19.1 Detailed Description	63
6.19.2 Member Function Documentation	64
6.19.2.1 random_Int()	64
6.20 RandomEvent Class Reference	64
6.20.1 Detailed Description	65
6.20.2 Member Function Documentation	65
6.20.2.1 apply()	65
6.20.2.2 getName()	66
6.21 TeutonicPanel Class Reference	66
6.21.1 Detailed Description	67
6.21.2 Constructor & Destructor Documentation	68
6.21.2.1 TeutonicPanel()	68
6.21.3 Member Function Documentation	68
6.21.3.1 setAlive()	68
6.21.3.2 setArchers()	68
6.21.3.3 setCavalrymans()	68
6.21.3.4 setDamage()	68
6.21.3.5 setKills()	69
6.21.3.6 setKnights()	69
6.22 UnitStats Struct Reference	69
6.22.1 Detailed Description	70
6.22.2 Member Data Documentation	70
6.22.2.1 attack	70
6.22.2.2 defense	70
6.22.2.3 hp	70
6.23 Warrior Class Reference	71
6.23.1 Detailed Description	72
6.23.2 Constructor & Destructor Documentation	73
6.23.2.1 Warrior()	73
6.23.3 Member Function Documentation	73
6.23.3.1 attack_Energy()	73
6.23.3.2 get_Attack()	73
6.23.3.3 get_Defense()	74
6.23.3.4 get_Health()	74
6.23.3.5 get_Type()	74

6.23.3.6 get_WeatherModifier()	74
6.23.3.7 is_Alive()	74
6.23.3.8 take_Damage()	75
6.24 Weather Class Reference	75
6.24.1 Detailed Description	76
6.24.2 Constructor & Destructor Documentation	76
6.24.2.1 Weather()	76
6.24.3 Member Function Documentation	76
6.24.3.1 get_WeatherType()	76
6.24.3.2 to_String()	76
7 File Documentation	77
7.1 ConfigLoader.hpp	77
7.2 FractionConfig.hpp	77
7.3 Army.hpp	78
7.4 Battle.hpp	78
7.5 BattleStats.hpp	79
7.6 UnitStats.hpp	80
7.7 AlliedHelp.hpp	80
7.8 CommanderDeath.hpp	80
7.9 PanicEvent.hpp	80
7.10 RandomEvent.hpp	81
7.11 Weather.hpp	81
7.12 WeatherType.hpp	81
7.13 BattlePanel.hpp	81
7.14 ControlPanel.hpp	82
7.15 HistoryPanel.hpp	83
7.16 HussarsPanel.hpp	83
7.17 MainWindow.hpp	83
7.18 TeutonicPanel.hpp	84
7.19 BattleObserver.hpp	85
7.20 BattleReport.hpp	85
7.21 Random.hpp	85
7.22 Archer.hpp	86
7.23 Cavalryman.hpp	86
7.24 Knight.hpp	86
7.25 Warrior.hpp	87
7.26 main.cpp	87
7.27 ConfigLoader.cpp	88
7.28 Army.cpp	91
7.29 Battle.cpp	92
7.30 BattleStats.cpp	95

7.31 AlliedHelp.cpp	95
7.32 CommanderDeath.cpp	95
7.33 PanicEvent.cpp	96
7.34 Weather.cpp	96
7.35 BattlePanel.cpp	97
7.36 ControlPanel.cpp	98
7.37 HistoryPanel.cpp	99
7.38 HussarsPanel.cpp	99
7.39 MainWindow.cpp	101
7.40 TeutonicPanel.cpp	105
7.41 BattleReport.cpp	107
7.42 Random.cpp	107
7.43 Archer.cpp	108
7.44 Cavalryman.cpp	108
7.45 Knight.cpp	109
7.46 Warrior.cpp	109
Index	111

Chapter 1

Simulation Battle of Grunwald

This documentation is generated with Doxygen from comments in the C++ source code.

1.1 Main Modules

- `core` - battle flow, armies and battle statistics.
- `warriors` - warrior hierarchy used during combat.
- `events` - random battle events and weather.
- `config` - CSV configuration loading.
- `gui` - Qt widgets used to run and display the simulation.
- `reports` - CSV battle report generation.

1.2 Generate Documentation

Run this command from the repository root:

```
doxygen Doxyfile
```

Generated HTML documentation is written to:

```
docs/doxygen/html/index.html
```

1.3 UML Diagrams

Additional UML diagrams are stored in `docs/UML`.

Chapter 2

Simulation Battle of Grunwald

Simulation of the [Battle](#) of Grunwald written in C++.

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Army	17
Battle	21
BattleObserver	25
BattlePanel	27
BattleReport	32
BattleStats	34
ConfigLoader	43
FractionConfig	47
QGroupBox	
HistoryPanel	50
QMainWindow	
MainWindow	60
QWidget	
BattlePanel	27
ControlPanel	44
HussarsPanel	52
TeutonicPanel	66
Random	63
RandomEvent	64
AlliedHelp	11
CommanderDeath	41
PanicEvent	61
UnitStats	69
Warrior	71
Archer	13
Cavalryman	36
Knight	55
Weather	75

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AlliedHelp	Random event that adds reinforcements and increases morale	11
Archer	Ranged unit with weather-sensitive attack effectiveness	13
Army	Represents one army taking part in a battle	17
Battle	Controls the simulation of one battle between two armies	21
BattleObserver	Observer interface used by GUI panels that react to battle updates	25
BattlePanel	GUI panel displaying battle-wide information	27
BattleReport	Writes battle results to CSV report files	32
BattleStats	Accumulates damage and kill counters for one battle	34
Cavalryman	Mounted unit with a stronger first attack	36
CommanderDeath	Random event that simulates commander death by lowering morale	41
ConfigLoader	Loads battle configuration rows from CSV files	43
ControlPanel	GUI panel with simulation control buttons and speed selector	44
FractionConfig	Stores all starting parameters needed to create one army fraction	47
HistoryPanel	GUI panel listing completed and skipped battles	50
HussarsPanel	GUI panel displaying current Hussars army statistics	52
Knight	Heavy infantry unit with attack and defense bonuses	55
MainWindow	Main Qt window coordinating configuration, simulation and reports	60
PanicEvent	Random event that decreases army morale	61

Random	Utility wrapper for generating random integer values	63
RandomEvent	Interface for random events that can affect an army during battle	64
TeutonicPanel	GUI panel displaying current Teutonic army statistics	66
UnitStats	Holds base statistics shared by every warrior type	69
Warrior	Abstract base class for all battle units	71
Weather	Represents weather selected for one battle	75

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

main.cpp	87
include/config/ConfigLoader.hpp	77
include/config/FractionConfig.hpp	77
include/core/Army.hpp	78
include/core/Battle.hpp	78
include/core/BattleStats.hpp	79
include/core/UnitStats.hpp	80
include/events/AlliedHelp.hpp	80
include/events/CommanderDeath.hpp	80
include/events/PanicEvent.hpp	80
include/events/RandomEvent.hpp	81
include/events/Weather.hpp	81
include/events/WeatherType.hpp	81
include/gui/BattlePanel.hpp	81
include/gui/ControlPanel.hpp	82
include/gui/HistoryPanel.hpp	83
include/gui/HussarsPanel.hpp	83
include/gui/MainWindow.hpp	83
include/gui/TeutonicPanel.hpp	84
include/observer/BattleObserver.hpp	85
include/reports/BattleReport.hpp	85
include/utils/Random.hpp	85
include/warriors/Archer.hpp	86
include/warriors/Cavalryman.hpp	86
include/warriors/Knight.hpp	86
include/warriors/Warrior.hpp	87
src/config/ConfigLoader.cpp	88
src/core/Army.cpp	91
src/core/Battle.cpp	92
src/core/BattleStats.cpp	95
src/events/AlliedHelp.cpp	95
src/events/CommanderDeath.cpp	95
src/events/PanicEvent.cpp	96
src/events/Weather.cpp	96
src/gui/BattlePanel.cpp	97

src/gui/ControlPanel.cpp	98
src/gui/HistoryPanel.cpp	99
src/gui/HussarsPanel.cpp	99
src/gui/MainWindow.cpp	101
src/gui/TeutonicPanel.cpp	105
src/reports/BattleReport.cpp	107
src/utls/Random.cpp	107
src/warriors/Archer.cpp	108
src/warriors/Cavalryman.cpp	108
src/warriors/Knight.cpp	109
src/warriors/Warrior.cpp	109

Chapter 6

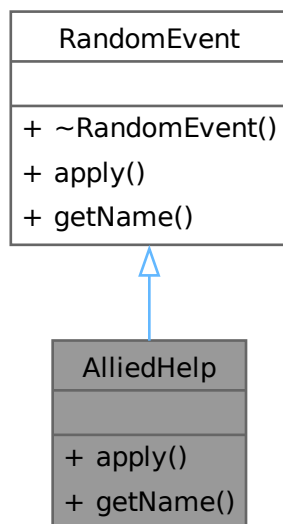
Class Documentation

6.1 AlliedHelp Class Reference

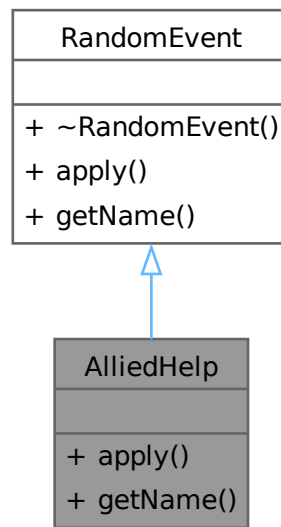
Random event that adds reinforcements and increases morale.

```
#include <AlliedHelp.hpp>
```

Inheritance diagram for AlliedHelp:



Collaboration diagram for AlliedHelp:



Public Member Functions

- void [apply](#) ([Army](#) &army) override
Adds warriors and morale to the affected army.
- std::string [getName](#) () const override
Returns the display name of the event.

Public Member Functions inherited from [RandomEvent](#)

- virtual `~RandomEvent ()`=default
Allows deleting derived event objects through a base pointer.

6.1.1 Detailed Description

[Random](#) event that adds reinforcements and increases morale.

Definition at line 12 of file [AlliedHelp.hpp](#).

6.1.2 Member Function Documentation

6.1.2.1 `apply()`

```
void AlliedHelp::apply (
    Army & army) [override], [virtual]
```

Adds warriors and morale to the affected army.

Implements [RandomEvent](#).

Definition at line 8 of file [AlliedHelp.cpp](#).

References [Army::add_Reinforcements\(\)](#), [Army::increase_Morale\(\)](#), and [Random::random_Int\(\)](#).

6.1.2.2 getName()

```
std::string AlliedHelp::getName () const [override], [virtual]
```

Returns the display name of the event.

Implements [RandomEvent](#).

Definition at line 13 of file [AlliedHelp.cpp](#).

The documentation for this class was generated from the following files:

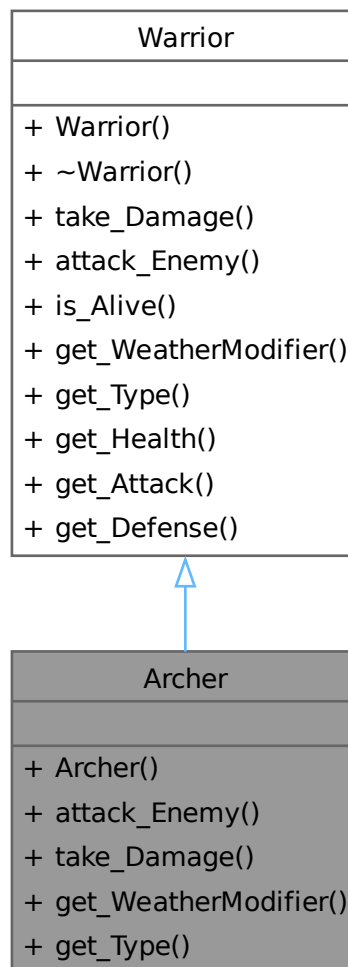
- include/events/AlliedHelp.hpp
- src/events/AlliedHelp.cpp

6.2 Archer Class Reference

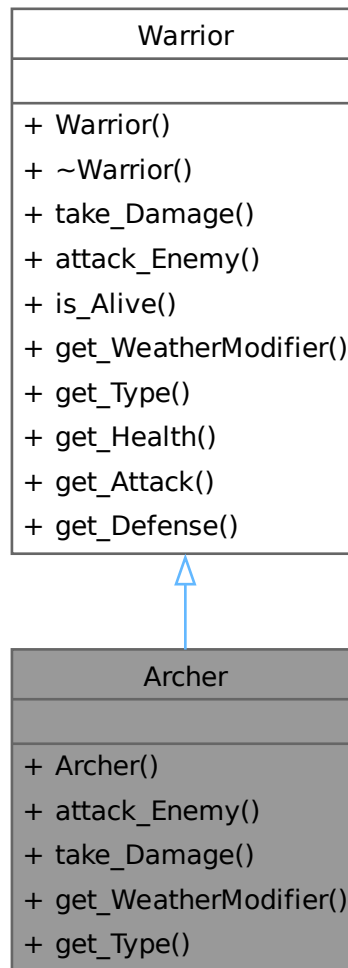
Ranged unit with weather-sensitive attack effectiveness.

```
#include <Archer.hpp>
```

Inheritance diagram for Archer:



Collaboration diagram for Archer:



Public Member Functions

- [Archer](#) (const [UnitStats](#) &stats)
Creates an archer from base statistics.
- void [attack_Enemy](#) ([Warrior](#) &enemy, double moraleModifier, WeatherType weather) override
Performs an archer attack against another warrior.
- void [take_Damage](#) (double damage) override
Applies incoming damage using archer defense behavior.
- double [get_WeatherModifier](#) (WeatherType weather) const override
Returns archer-specific weather modifier.
- std::string [get_Type](#) () const override
Returns "Archer".

Public Member Functions inherited from [Warrior](#)

- [Warrior](#) ([UnitStats](#))
Creates a warrior from base unit statistics.
- virtual `~Warrior ()`=default
Enables safe destruction through a base pointer.
- bool [is_Alive](#) () const
Checks whether the warrior can still fight.
- int [get_Health](#) () const
Returns current health.
- int [get_Attack](#) () const
Returns base attack value.
- int [get_Defense](#) () const
Returns base defense value.

6.2.1 Detailed Description

Ranged unit with weather-sensitive attack effectiveness.

Definition at line 13 of file [Archer.hpp](#).

6.2.2 Constructor & Destructor Documentation

6.2.2.1 [Archer\(\)](#)

```
Archer::Archer (
    const UnitStats & stats) [explicit]
```

Creates an archer from base statistics.

Definition at line 12 of file [Archer.cpp](#).

References [Warrior::Warrior\(\)](#).

6.2.3 Member Function Documentation

6.2.3.1 [attack_Enemy\(\)](#)

```
void Archer::attack_Enemy (
    Warrior & enemy,
    double moraleModifier,
    WeatherType weather) [override], [virtual]
```

Performs an archer attack against another warrior.

Implements [Warrior](#).

Definition at line 33 of file [Archer.cpp](#).

References [Warrior::get_Attack\(\)](#), [Warrior::get_Defense\(\)](#), [get_WeatherModifier\(\)](#), [Random::random_Int\(\)](#), [Warrior::take_Damage\(\)](#), and [Warrior::Warrior\(\)](#).

6.2.3.2 `get_Type()`

```
std::string Archer::get_Type () const [inline], [override], [virtual]
```

Returns "Archer".

Implements [Warrior](#).

Definition at line 32 of file [Archer.hpp](#).

6.2.3.3 `get_WeatherModifier()`

```
double Archer::get_WeatherModifier (  
    WeatherType weather) const [override], [virtual]
```

Returns archer-specific weather modifier.

Implements [Warrior](#).

Definition at line 18 of file [Archer.cpp](#).

Referenced by [attack_Enemy\(\)](#).

6.2.3.4 `take_Damage()`

```
void Archer::take_Damage (  
    double damage) [override], [virtual]
```

Applies incoming damage using archer defense behavior.

Reimplemented from [Warrior](#).

Definition at line 45 of file [Archer.cpp](#).

References [Warrior::take_Damage\(\)](#).

The documentation for this class was generated from the following files:

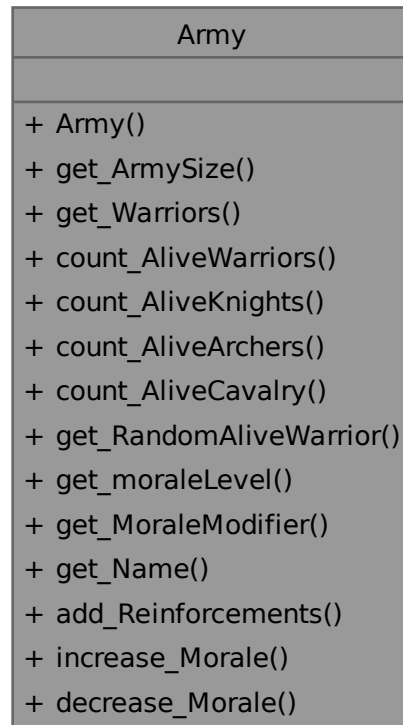
- `include/warriors/Archer.hpp`
- `src/warriors/Archer.cpp`

6.3 Army Class Reference

Represents one army taking part in a battle.

```
#include <Army.hpp>
```

Collaboration diagram for Army:



Public Member Functions

- `Army` (const `FractionConfig` &config)
Creates an army from loaded fraction configuration.
- `std::size_t get_ArmySize ()` const
Returns total number of warriors in the army.
- `const std::vector< std::shared_ptr< Warrior > > & get_Warriors ()` const
Returns all warriors belonging to the army.
- `int count_AliveWarriors ()` const
Counts warriors that are still alive.
- `int count_AliveKnights ()` const
Counts living knights.
- `int count_AliveArchers ()` const
Counts living archers.
- `int count_AliveCavalry ()` const

- Counts living cavalrymen.*
 - [Warrior](#) & [get_RandomAliveWarrior](#) ()
 - Selects a random living warrior.*
- int [get_moraleLevel](#) () const
 - Returns current morale level.*
- double [get_MoraleModifier](#) () const
 - Returns morale modifier used in attack calculations.*
- const std::string & [get_Name](#) () const
 - Returns army name.*
- void [add_Reinforcements](#) (int amount)
 - Adds new warriors as reinforcements.*
- void [increase_Morale](#) (int amount)
 - Increases morale by the given amount.*
- void [decrease_Morale](#) (int amount)
 - Decreases morale by the given amount.*

6.3.1 Detailed Description

Represents one army taking part in a battle.

[Army](#) owns a collection of warriors created from a fraction configuration and tracks morale used during combat.

Definition at line 23 of file [Army.hpp](#).

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Army()

```
Army::Army (
    const FractionConfig & config)
```

Creates an army from loaded fraction configuration.

Exceptions

<code>std::runtime_error</code>	when the configuration is invalid.
---------------------------------	------------------------------------

Definition at line 13 of file [Army.cpp](#).

References [FractionConfig::archer_count](#), [FractionConfig::archer_stats](#), [FractionConfig::cavalry_count](#), [FractionConfig::cavalry_stats](#), [FractionConfig::knight_count](#), [FractionConfig::knight_stats](#), and [FractionConfig::valid](#).

6.3.3 Member Function Documentation

6.3.3.1 add_Reinforcements()

```
void Army::add_Reinforcements (
    int amount)
```

Adds new warriors as reinforcements.

Definition at line 121 of file [Army.cpp](#).

References [Random::random_Int](#)().

Referenced by [AlliedHelp::apply](#)().

6.3.3.2 count_AliveArchers()

```
int Army::count_AliveArchers () const
```

Counts living archers.

Definition at line 59 of file [Army.cpp](#).

6.3.3.3 count_AliveCavalry()

```
int Army::count_AliveCavalry () const
```

Counts living cavalymen.

Definition at line 70 of file [Army.cpp](#).

6.3.3.4 count_AliveKnights()

```
int Army::count_AliveKnights () const
```

Counts living knights.

Definition at line 48 of file [Army.cpp](#).

6.3.3.5 count_AliveWarriors()

```
int Army::count_AliveWarriors () const
```

Counts warriors that are still alive.

Definition at line 38 of file [Army.cpp](#).

6.3.3.6 decrease_Morale()

```
void Army::decrease_Morale (  
    int amount)
```

Decreases morale by the given amount.

Definition at line 115 of file [Army.cpp](#).

Referenced by [CommanderDeath::apply\(\)](#), and [PanicEvent::apply\(\)](#).

6.3.3.7 get_ArmySize()

```
size_t Army::get_ArmySize () const
```

Returns total number of warriors in the army.

Definition at line 83 of file [Army.cpp](#).

6.3.3.8 get_moraleLevel()

```
int Army::get_moraleLevel () const
```

Returns current morale level.

Definition at line 141 of file [Army.cpp](#).

6.3.3.9 get_MoraleModifier()

```
double Army::get_MoraleModifier () const
```

Returns morale modifier used in attack calculations.

Definition at line 145 of file [Army.cpp](#).

6.3.3.10 get_Name()

```
const std::string & Army::get_Name () const
```

Returns army name.

Definition at line 156 of file [Army.cpp](#).

6.3.3.11 get_RandomAliveWarrior()

```
Warrior & Army::get_RandomAliveWarrior ()
```

Selects a random living warrior.

Returns

Reference to selected warrior.

Definition at line 91 of file [Army.cpp](#).

References [Random::random_Int\(\)](#).

6.3.3.12 get_Warriors()

```
const std::vector< std::shared_ptr< Warrior > > & Army::get_Warriors () const
```

Returns all warriors belonging to the army.

Definition at line 87 of file [Army.cpp](#).

6.3.3.13 increase_Morale()

```
void Army::increase_Morale (
    int amount)
```

Increases morale by the given amount.

Definition at line 110 of file [Army.cpp](#).

Referenced by [AlliedHelp::apply\(\)](#).

The documentation for this class was generated from the following files:

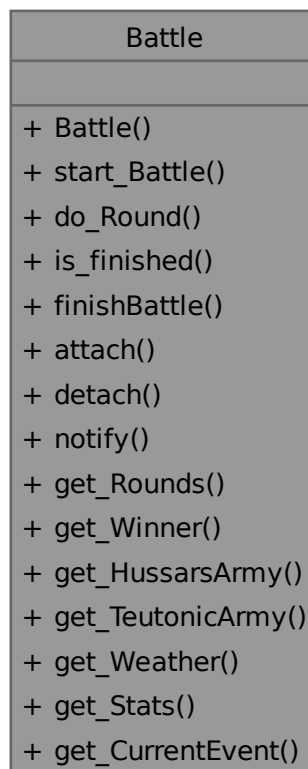
- include/core/Army.hpp
- src/core/Army.cpp

6.4 Battle Class Reference

Controls the simulation of one battle between two armies.

```
#include <Battle.hpp>
```

Collaboration diagram for Battle:



Public Member Functions

- [Battle](#) ([Army](#) &[HussarsArmy](#), [Army](#) &[TeutonicArmy](#))
Creates a battle from two prepared armies.
- void [start_Battle](#) ()
Starts the battle loop in non-GUI usage.
- void [do_Round](#) ()
Executes one combat round and notifies observers.
- bool [is_finished](#) () const
Checks whether one army has no living warriors.
- void [finishBattle](#) ()
Determines and stores the final winner.
- void [attach](#) ([BattleObserver](#) *observer)
Registers an observer notified after battle updates.
- void [detach](#) ([BattleObserver](#) *observer)
Unregisters an observer.
- void [notify](#) ()
Calls update on all registered observers.
- int [get_Rounds](#) () const
Returns current round number.
- std::string [get_Winner](#) () const
Returns winner name after the battle is finished.
- const [Army](#) & [get_HussarsArmy](#) () const
Returns the Hussars army state.
- const [Army](#) & [get_TeutonicArmy](#) () const
Returns the Teutonic army state.
- std::string [get_Weather](#) () const
Returns weather name as text.
- const [BattleStats](#) & [get_Stats](#) () const
Returns accumulated battle statistics.
- const std::string & [get_CurrentEvent](#) () const
Returns text describing the last combat action or random event.

6.4.1 Detailed Description

Controls the simulation of one battle between two armies.

[Battle](#) owns copied army instances, tracks weather, statistics and random events, and notifies registered observers after state changes.

Definition at line 20 of file [Battle.hpp](#).

6.4.2 Constructor & Destructor Documentation

6.4.2.1 Battle()

```
Battle::Battle (
    Army & HussarsArmy,
    Army & TeutonicArmy)
```

Creates a battle from two prepared armies.

Definition at line 13 of file [Battle.cpp](#).

6.4.3 Member Function Documentation

6.4.3.1 attach()

```
void Battle::attach (  
    BattleObserver * observer)
```

Registers an observer notified after battle updates.

Definition at line 156 of file [Battle.cpp](#).

6.4.3.2 do_Round()

```
void Battle::do_Round ()
```

Executes one combat round and notifies observers.

Definition at line 33 of file [Battle.cpp](#).

References [Warrior::attack_Enemy\(\)](#), [Warrior::get_Health\(\)](#), [Warrior::get_Type\(\)](#), [Warrior::is_Alive\(\)](#), [notify\(\)](#), and [Random::random_Int\(\)](#).

Referenced by [start_Battle\(\)](#).

6.4.3.3 finishBattle()

```
void Battle::finishBattle ()
```

Determines and stores the final winner.

Definition at line 116 of file [Battle.cpp](#).

Referenced by [start_Battle\(\)](#).

6.4.3.4 get_CurrentEvent()

```
const std::string & Battle::get_CurrentEvent () const
```

Returns text describing the last combat action or random event.

Definition at line 160 of file [Battle.cpp](#).

6.4.3.5 get_HussarsArmy()

```
const Army & Battle::get_HussarsArmy () const
```

Returns the Hussars army state.

Definition at line 140 of file [Battle.cpp](#).

6.4.3.6 get_Rounds()

```
int Battle::get_Rounds () const
```

Returns current round number.

Definition at line 132 of file [Battle.cpp](#).

6.4.3.7 get_Stats()

```
const BattleStats & Battle::get_Stats () const
```

Returns accumulated battle statistics.

Definition at line 152 of file [Battle.cpp](#).

6.4.3.8 get_TeutonicArmy()

```
const Army & Battle::get_TeutonicArmy () const
```

Returns the Teutonic army state.

Definition at line 144 of file [Battle.cpp](#).

6.4.3.9 get_Weather()

```
std::string Battle::get_Weather () const
```

Returns weather name as text.

Definition at line 148 of file [Battle.cpp](#).

6.4.3.10 get_Winner()

```
std::string Battle::get_Winner () const
```

Returns winner name after the battle is finished.

Definition at line 136 of file [Battle.cpp](#).

6.4.3.11 is_finished()

```
bool Battle::is_finished () const
```

Checks whether one army has no living warriors.

Definition at line 112 of file [Battle.cpp](#).

Referenced by [start_Battle\(\)](#).

6.4.3.12 notify()

```
void Battle::notify ()
```

Calls update on all registered observers.

Definition at line 125 of file [Battle.cpp](#).

Referenced by [do_Round\(\)](#).

6.4.3.13 start_Battle()

```
void Battle::start_Battle ()
```

Starts the battle loop in non-GUI usage.

Definition at line 24 of file [Battle.cpp](#).

References [do_Round\(\)](#), [finishBattle\(\)](#), and [is_finished\(\)](#).

The documentation for this class was generated from the following files:

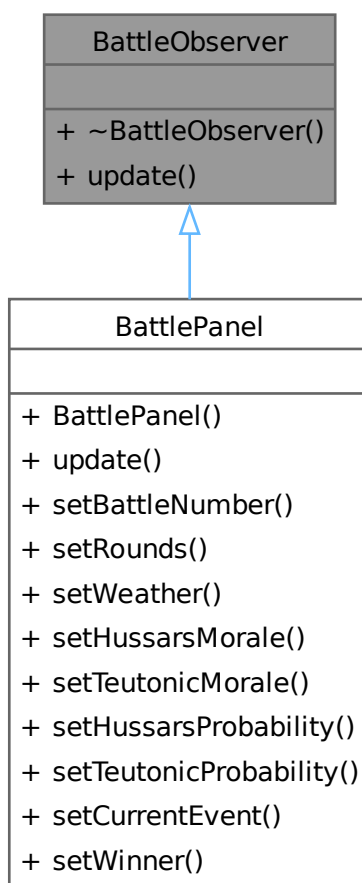
- [include/core/Battle.hpp](#)
- [src/core/Battle.cpp](#)

6.5 BattleObserver Class Reference

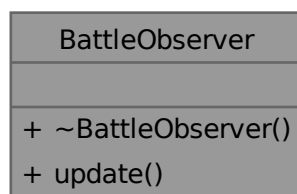
Observer interface used by GUI panels that react to battle updates.

```
#include <BattleObserver.hpp>
```

Inheritance diagram for BattleObserver:



Collaboration diagram for BattleObserver:



Public Member Functions

- virtual `~BattleObserver()`=default

Allows deleting observers through a base pointer.

- virtual void [update](#) (const [Battle](#) &battle)=0

Receives the current battle state.

6.5.1 Detailed Description

Observer interface used by GUI panels that react to battle updates.

Definition at line 13 of file [BattleObserver.hpp](#).

6.5.2 Member Function Documentation

6.5.2.1 update()

```
virtual void BattleObserver::update (  
    const Battle & battle) [pure virtual]
```

Receives the current battle state.

Parameters

<i>battle</i>	Battle that changed.
---------------	--------------------------------------

Implemented in [BattlePanel](#).

The documentation for this class was generated from the following file:

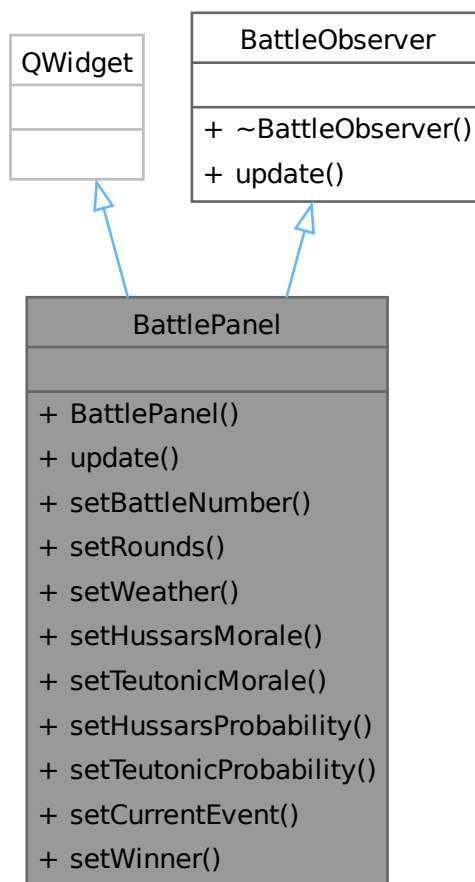
- include/observer/BattleObserver.hpp

6.6 BattlePanel Class Reference

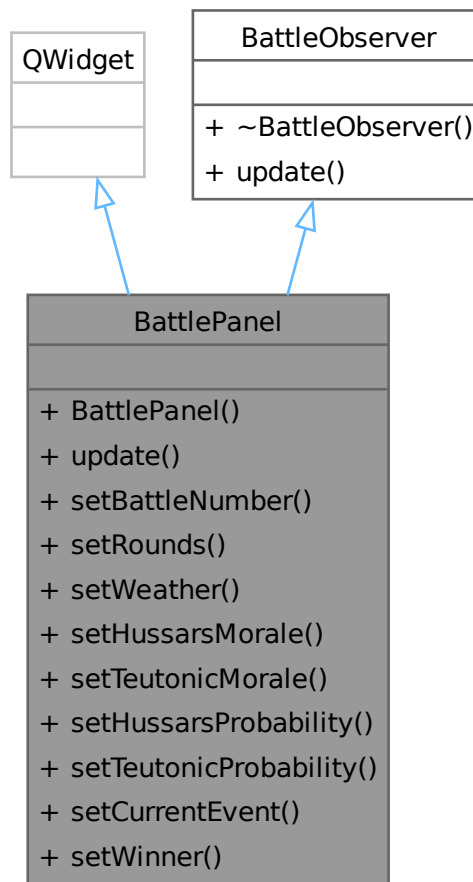
GUI panel displaying battle-wide information.

```
#include <BattlePanel.hpp>
```

Inheritance diagram for BattlePanel:



Collaboration diagram for BattlePanel:



Public Member Functions

- **BattlePanel** (QWidget *parent=nullptr)
Creates labels used to display current battle state.
- void **update** (const **Battle** &battle) override
Updates battle information after observer notification.
- void **setBattleNumber** (int current, int total)
Displays current battle number.
- void **setRounds** (int rounds)
Displays current round count.
- void **setWeather** (const QString &weather)
Displays current weather.
- void **setHussarsMorale** (int morale)
Displays Hussars morale.
- void **setTeutonicMorale** (int morale)
Displays Teutonic morale.
- void **setHussarsProbability** (double probability)

- *Displays estimated Hussars victory probability.*
void [setTeutonicProbability](#) (double probability)
- *Displays estimated Teutonic victory probability.*
void [setCurrentEvent](#) (const QString &event)
- *Displays the latest combat action or random event.*
void [setWinner](#) (const QString &winner)
- *Displays the currently predicted or final winner.*

Public Member Functions inherited from [BattleObserver](#)

- virtual [~BattleObserver](#) ()=default
Allows deleting observers through a base pointer.

6.6.1 Detailed Description

GUI panel displaying battle-wide information.

Definition at line 18 of file [BattlePanel.hpp](#).

6.6.2 Constructor & Destructor Documentation

6.6.2.1 BattlePanel()

```
BattlePanel::BattlePanel (
    QWidget * parent = nullptr)
```

Creates labels used to display current battle state.

Definition at line 28 of file [BattlePanel.cpp](#).

6.6.3 Member Function Documentation

6.6.3.1 setBattleNumber()

```
void BattlePanel::setBattleNumber (
    int current,
    int total)
```

Displays current battle number.

Definition at line 89 of file [BattlePanel.cpp](#).

6.6.3.2 setCurrentEvent()

```
void BattlePanel::setCurrentEvent (
    const QString & event)
```

Displays the latest combat action or random event.

Definition at line 134 of file [BattlePanel.cpp](#).

Referenced by [update\(\)](#).

6.6.3.3 setHussarsMorale()

```
void BattlePanel::setHussarsMorale (
    int morale)
```

Displays Hussars morale.

Definition at line 104 of file [BattlePanel.cpp](#).

Referenced by [update\(\)](#).

6.6.3.4 setHussarsProbability()

```
void BattlePanel::setHussarsProbability (
    double probability)
```

Displays estimated Hussars victory probability.

Definition at line 124 of file [BattlePanel.cpp](#).

6.6.3.5 setRounds()

```
void BattlePanel::setRounds (
    int rounds)
```

Displays current round count.

Definition at line 97 of file [BattlePanel.cpp](#).

Referenced by [update\(\)](#).

6.6.3.6 setTeutonicMorale()

```
void BattlePanel::setTeutonicMorale (
    int morale)
```

Displays Teutonic morale.

Definition at line 114 of file [BattlePanel.cpp](#).

Referenced by [update\(\)](#).

6.6.3.7 setTeutonicProbability()

```
void BattlePanel::setTeutonicProbability (
    double probability)
```

Displays estimated Teutonic victory probability.

Definition at line 129 of file [BattlePanel.cpp](#).

6.6.3.8 setWeather()

```
void BattlePanel::setWeather (
    const QString & weather)
```

Displays current weather.

Definition at line 100 of file [BattlePanel.cpp](#).

Referenced by [update\(\)](#).

6.6.3.9 setWinner()

```
void BattlePanel::setWinner (
    const QString & winner)
```

Displays the currently predicted or final winner.

Definition at line 138 of file [BattlePanel.cpp](#).

6.6.3.10 update()

```
void BattlePanel::update (
    const Battle & battle) [override], [virtual]
```

Updates battle information after observer notification.

Implements [BattleObserver](#).

Definition at line 15 of file [BattlePanel.cpp](#).

References [setCurrentEvent\(\)](#), [setHussarsMorale\(\)](#), [setRounds\(\)](#), [setTeutonicMorale\(\)](#), and [setWeather\(\)](#).

The documentation for this class was generated from the following files:

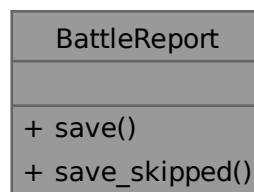
- include/gui/BattlePanel.hpp
- src/gui/BattlePanel.cpp

6.7 BattleReport Class Reference

Writes battle results to CSV report files.

```
#include <BattleReport.hpp>
```

Collaboration diagram for BattleReport:



Static Public Member Functions

- static void [save](#) (const [Battle](#) &battle, const std::string &filename, int battleNumber)
Appends a completed battle result to a CSV report.
- static void [save_skipped](#) (const std::string &filename, int battleNumber, const std::string &reason)
Appends information about a skipped battle to a CSV report.

6.7.1 Detailed Description

Writes battle results to CSV report files.

Definition at line 14 of file [BattleReport.hpp](#).

6.7.2 Member Function Documentation

6.7.2.1 [save\(\)](#)

```
void BattleReport::save (  
    const Battle & battle,  
    const std::string & filename,  
    int battleNumber) [static]
```

Appends a completed battle result to a CSV report.

Definition at line 11 of file [BattleReport.cpp](#).

6.7.2.2 [save_skipped\(\)](#)

```
void BattleReport::save_skipped (  
    const std::string & filename,  
    int battleNumber,  
    const std::string & reason) [static]
```

Appends information about a skipped battle to a CSV report.

Definition at line 34 of file [BattleReport.cpp](#).

The documentation for this class was generated from the following files:

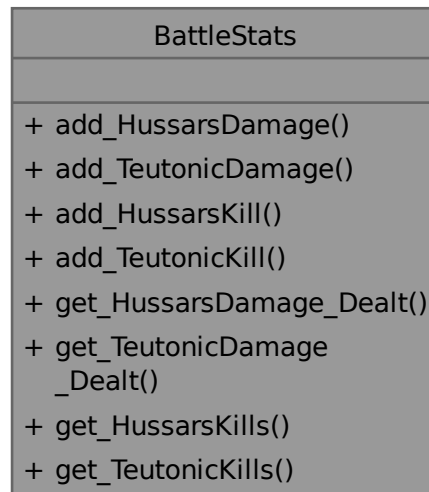
- include/reports/BattleReport.hpp
- src/reports/BattleReport.cpp

6.8 BattleStats Class Reference

Accumulates damage and kill counters for one battle.

```
#include <BattleStats.hpp>
```

Collaboration diagram for BattleStats:



Public Member Functions

- void [add_HussarsDamage](#) (double damage)
Adds damage dealt by the Hussars army.
- void [add_TeutonicDamage](#) (double damage)
Adds damage dealt by the Teutonic army.
- void [add_HussarsKill](#) ()
Increments Hussars kill counter.
- void [add_TeutonicKill](#) ()
Increments Teutonic kill counter.
- double [get_HussarsDamage_Dealt](#) () const
Returns total damage dealt by Hussars.
- double [get_TeutonicDamage_Dealt](#) () const
Returns total damage dealt by Teutonic army.
- int [get_HussarsKills](#) () const
Returns number of kills made by Hussars.
- int [get_TeutonicKills](#) () const
Returns number of kills made by Teutonic army.

6.8.1 Detailed Description

Accumulates damage and kill counters for one battle.

Definition at line 11 of file [BattleStats.hpp](#).

6.8.2 Member Function Documentation

6.8.2.1 add_HussarsDamage()

```
void BattleStats::add_HussarsDamage (
    double damage)
```

Adds damage dealt by the Hussars army.

Definition at line 9 of file [BattleStats.cpp](#).

6.8.2.2 add_HussarsKill()

```
void BattleStats::add_HussarsKill ()
```

Increments Hussars kill counter.

Definition at line 18 of file [BattleStats.cpp](#).

6.8.2.3 add_TeutonicDamage()

```
void BattleStats::add_TeutonicDamage (
    double damage)
```

Adds damage dealt by the Teutonic army.

Definition at line 13 of file [BattleStats.cpp](#).

6.8.2.4 add_TeutonicKill()

```
void BattleStats::add_TeutonicKill ()
```

Increments Teutonic kill counter.

Definition at line 22 of file [BattleStats.cpp](#).

6.8.2.5 get_HussarsDamage_Dealt()

```
double BattleStats::get_HussarsDamage_Dealt () const
```

Returns total damage dealt by Hussars.

Definition at line 29 of file [BattleStats.cpp](#).

6.8.2.6 `get_HussarsKills()`

```
int BattleStats::get_HussarsKills () const
```

Returns number of kills made by Hussars.

Definition at line 38 of file [BattleStats.cpp](#).

6.8.2.7 `get_TeutonicDamage_Dealt()`

```
double BattleStats::get_TeutonicDamage_Dealt () const
```

Returns total damage dealt by Teutonic army.

Definition at line 33 of file [BattleStats.cpp](#).

6.8.2.8 `get_TeutonicKills()`

```
int BattleStats::get_TeutonicKills () const
```

Returns number of kills made by Teutonic army.

Definition at line 42 of file [BattleStats.cpp](#).

The documentation for this class was generated from the following files:

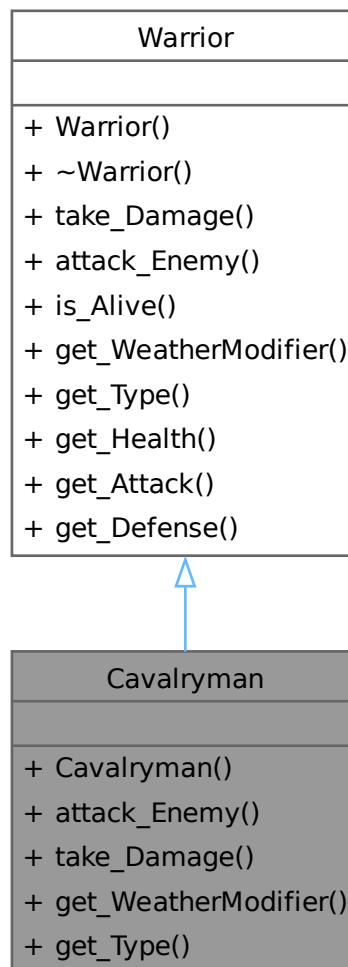
- [include/core/BattleStats.hpp](#)
- [src/core/BattleStats.cpp](#)

6.9 Cavalryman Class Reference

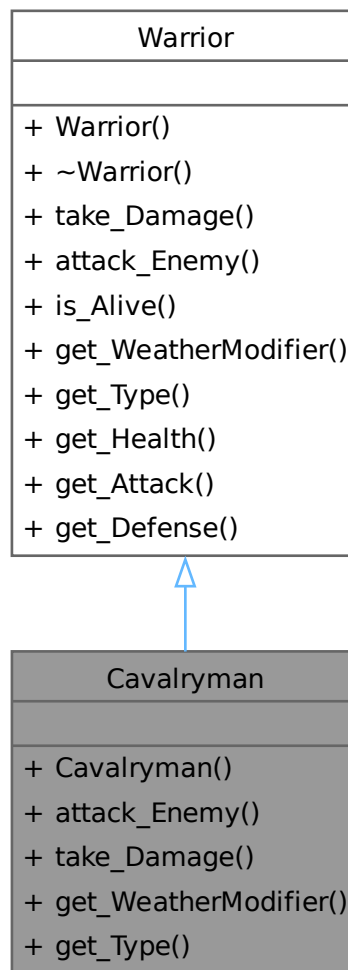
Mounted unit with a stronger first attack.

```
#include <Cavalryman.hpp>
```

Inheritance diagram for Cavalryman:



Collaboration diagram for Cavalryman:



Public Member Functions

- **Cavalryman** (const **UnitStats** &stats)
Creates a cavalryman from base statistics.
- void **attack_Enemy** (**Warrior** &enemy, double moraleModifier, WeatherType weather) override
Performs a cavalry attack against another warrior.
- void **take_Damage** (double damage) override
Applies incoming damage using cavalry defense behavior.
- double **get_WeatherModifier** (WeatherType weather) const override
Returns cavalry-specific weather modifier.
- std::string **get_Type** () const override
Returns "Cavalryman".

Public Member Functions inherited from [Warrior](#)

- [Warrior](#) ([UnitStats](#))
Creates a warrior from base unit statistics.
- virtual `~Warrior ()`=default
Enables safe destruction through a base pointer.
- bool [is_Alive](#) () const
Checks whether the warrior can still fight.
- int [get_Health](#) () const
Returns current health.
- int [get_Attack](#) () const
Returns base attack value.
- int [get_Defense](#) () const
Returns base defense value.

6.9.1 Detailed Description

Mounted unit with a stronger first attack.

Definition at line 13 of file [Cavalryman.hpp](#).

6.9.2 Constructor & Destructor Documentation

6.9.2.1 Cavalryman()

```
Cavalryman::Cavalryman (  
    const UnitStats & stats) [explicit]
```

Creates a cavalryman from base statistics.

Definition at line 11 of file [Cavalryman.cpp](#).

References [Warrior::Warrior\(\)](#).

6.9.3 Member Function Documentation

6.9.3.1 attack_Enemy()

```
void Cavalryman::attack_Enemy (  
    Warrior & enemy,  
    double moraleModifier,  
    WeatherType weather) [override], [virtual]
```

Performs a cavalry attack against another warrior.

Implements [Warrior](#).

Definition at line 33 of file [Cavalryman.cpp](#).

References [Warrior::get_Attack\(\)](#), [Warrior::get_Defense\(\)](#), [get_WeatherModifier\(\)](#), [Warrior::take_Damage\(\)](#), and [Warrior::Warrior\(\)](#).

6.9.3.2 `get_Type()`

```
std::string Cavalryman::get_Type () const [inline], [override], [virtual]
```

Returns "Cavalryman".

Implements [Warrior](#).

Definition at line 33 of file [Cavalryman.hpp](#).

6.9.3.3 `get_WeatherModifier()`

```
double Cavalryman::get_WeatherModifier (  
    WeatherType weather) const [override], [virtual]
```

Returns cavalry-specific weather modifier.

Implements [Warrior](#).

Definition at line 18 of file [Cavalryman.cpp](#).

Referenced by [attack_Enemy\(\)](#).

6.9.3.4 `take_Damage()`

```
void Cavalryman::take_Damage (  
    double damage) [override], [virtual]
```

Applies incoming damage using cavalry defense behavior.

Reimplemented from [Warrior](#).

Definition at line 46 of file [Cavalryman.cpp](#).

References [Warrior::take_Damage\(\)](#).

The documentation for this class was generated from the following files:

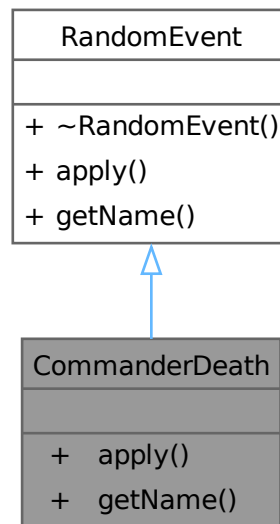
- [include/warriors/Cavalryman.hpp](#)
- [src/warriors/Cavalryman.cpp](#)

6.10 CommanderDeath Class Reference

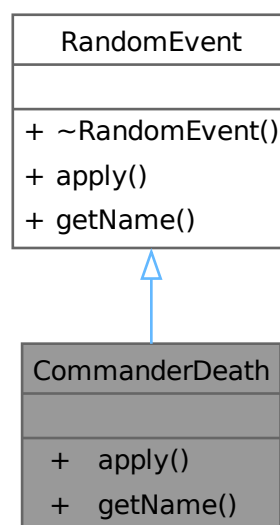
[Random](#) event that simulates commander death by lowering morale.

```
#include <CommanderDeath.hpp>
```

Inheritance diagram for CommanderDeath:



Collaboration diagram for CommanderDeath:



Public Member Functions

- void [apply](#) ([Army](#) &army) override
Decreases morale of the affected army.
- std::string [getName](#) () const override
Returns the display name of the event.

Public Member Functions inherited from [RandomEvent](#)

- virtual [~RandomEvent](#) ()=default
Allows deleting derived event objects through a base pointer.

6.10.1 Detailed Description

[Random](#) event that simulates commander death by lowering morale.

Definition at line 12 of file [CommanderDeath.hpp](#).

6.10.2 Member Function Documentation

6.10.2.1 [apply\(\)](#)

```
void CommanderDeath::apply (  
    Army & army) [override], [virtual]
```

Decreases morale of the affected army.

Implements [RandomEvent](#).

Definition at line 7 of file [CommanderDeath.cpp](#).

References [Army::decrease_Morale\(\)](#).

6.10.2.2 [getName\(\)](#)

```
std::string CommanderDeath::getName () const [override], [virtual]
```

Returns the display name of the event.

Implements [RandomEvent](#).

Definition at line 11 of file [CommanderDeath.cpp](#).

The documentation for this class was generated from the following files:

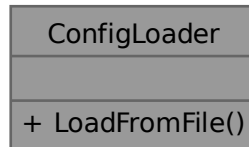
- include/events/CommanderDeath.hpp
- src/events/CommanderDeath.cpp

6.11 ConfigLoader Class Reference

Loads battle configuration rows from CSV files.

```
#include <ConfigLoader.hpp>
```

Collaboration diagram for ConfigLoader:



Static Public Member Functions

- static `std::vector< FractionConfig > LoadFromFile` (const `std::string &filename`, bool `printErrors`)
Reads fraction configurations from a CSV file.

6.11.1 Detailed Description

Loads battle configuration rows from CSV files.

Definition at line 14 of file [ConfigLoader.hpp](#).

6.11.2 Member Function Documentation

6.11.2.1 LoadFromFile()

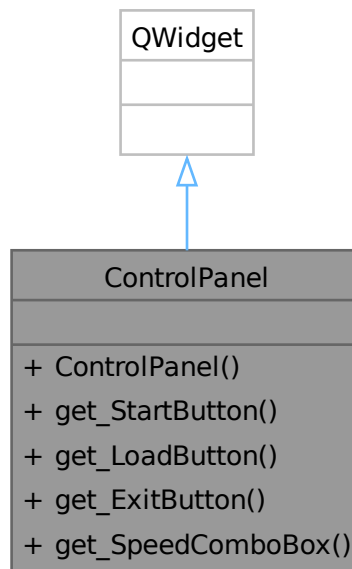
```
std::vector< FractionConfig > ConfigLoader::LoadFromFile (  
    const std::string & filename,  
    bool printErrors = true) [static]
```

Reads fraction configurations from a CSV file.

Parameters

<i>filename</i>	Path to the CSV configuration file.
<i>printErrors</i>	If true, validation errors are printed to stderr/stdout.

Collaboration diagram for ControlPanel:



Public Member Functions

- `ControlPanel` (`QWidget *parent`)
Creates buttons for loading, starting, exiting and selecting speed.
- `QPushButton * get_StartButton ()` const
Returns the Start button.
- `QPushButton * get_LoadButton ()` const
Returns the Load button.
- `QPushButton * get_ExitButton ()` const
Returns the Exit button.
- `QComboBox * get_SpeedComboBox ()` const
Returns the speed selection combo box.

6.12.1 Detailed Description

GUI panel with simulation control buttons and speed selector.

Definition at line 15 of file [ControlPanel.hpp](#).

6.12.2 Constructor & Destructor Documentation

6.12.2.1 ControlPanel()

```
ControlPanel::ControlPanel (
    QWidget * parent)
```

Creates buttons for loading, starting, exiting and selecting speed.

Definition at line 9 of file [ControlPanel.cpp](#).

6.12.3 Member Function Documentation

6.12.3.1 `get_ExitButton()`

```
QPushButton * ControlPanel::get_ExitButton () const
```

Returns the Exit button.

Definition at line 46 of file [ControlPanel.cpp](#).

6.12.3.2 `get_LoadButton()`

```
QPushButton * ControlPanel::get_LoadButton () const
```

Returns the Load button.

Definition at line 42 of file [ControlPanel.cpp](#).

6.12.3.3 `get_SpeedComboBox()`

```
QComboBox * ControlPanel::get_SpeedComboBox () const
```

Returns the speed selection combo box.

Definition at line 50 of file [ControlPanel.cpp](#).

6.12.3.4 `get_StartButton()`

```
QPushButton * ControlPanel::get_StartButton () const
```

Returns the Start button.

Definition at line 38 of file [ControlPanel.cpp](#).

The documentation for this class was generated from the following files:

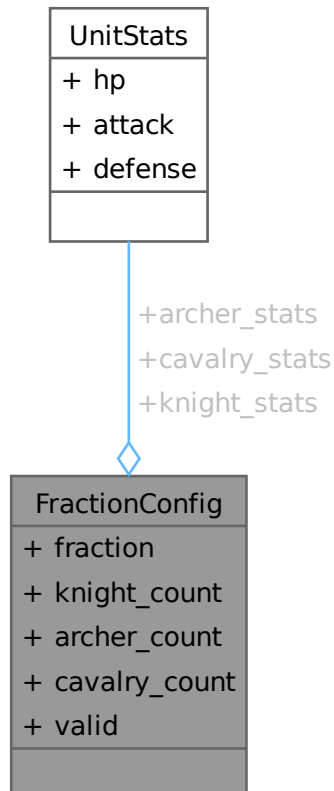
- `include/gui/ControlPanel.hpp`
- `src/gui/ControlPanel.cpp`

6.13 FractionConfig Struct Reference

Stores all starting parameters needed to create one army fraction.

```
#include <FractionConfig.hpp>
```

Collaboration diagram for FractionConfig:



Public Attributes

- `std::string` [fraction](#)
Fraction name displayed in reports and GUI.
- `UnitStats` [knight_stats](#)
Base statistics for knights.
- `int` [knight_count](#)
Number of knights created for the army.
- `UnitStats` [archer_stats](#)
Base statistics for archers.
- `int` [archer_count](#)
Number of archers created for the army.
- `UnitStats` [cavalry_stats](#)

Base statistics for cavalrymen.

- int [cavalry_count](#)

Number of cavalrymen created for the army.

- bool [valid](#) = true

Indicates whether the row loaded from configuration passed validation.

6.13.1 Detailed Description

Stores all starting parameters needed to create one army fraction.

Definition at line 14 of file [FractionConfig.hpp](#).

6.13.2 Member Data Documentation

6.13.2.1 archer_count

```
int FractionConfig::archer_count
```

Number of archers created for the army.

Definition at line 26 of file [FractionConfig.hpp](#).

Referenced by [Army::Army\(\)](#), and [ConfigLoader::LoadFromFile\(\)](#).

6.13.2.2 archer_stats

```
UnitStats FractionConfig::archer_stats
```

Base statistics for archers.

Definition at line 24 of file [FractionConfig.hpp](#).

Referenced by [Army::Army\(\)](#), and [ConfigLoader::LoadFromFile\(\)](#).

6.13.2.3 cavalry_count

```
int FractionConfig::cavalry_count
```

Number of cavalrymen created for the army.

Definition at line 31 of file [FractionConfig.hpp](#).

Referenced by [Army::Army\(\)](#), and [ConfigLoader::LoadFromFile\(\)](#).

6.13.2.4 cavalry_stats

`UnitStats FractionConfig::cavalry_stats`

Base statistics for cavalrymen.

Definition at line 29 of file [FractionConfig.hpp](#).

Referenced by [Army::Army\(\)](#), and [ConfigLoader::LoadFromFile\(\)](#).

6.13.2.5 fraction

`std::string FractionConfig::fraction`

Fraction name displayed in reports and GUI.

Definition at line 16 of file [FractionConfig.hpp](#).

Referenced by [ConfigLoader::LoadFromFile\(\)](#).

6.13.2.6 knight_count

`int FractionConfig::knight_count`

Number of knights created for the army.

Definition at line 21 of file [FractionConfig.hpp](#).

Referenced by [Army::Army\(\)](#), and [ConfigLoader::LoadFromFile\(\)](#).

6.13.2.7 knight_stats

`UnitStats FractionConfig::knight_stats`

Base statistics for knights.

Definition at line 19 of file [FractionConfig.hpp](#).

Referenced by [Army::Army\(\)](#), and [ConfigLoader::LoadFromFile\(\)](#).

6.13.2.8 valid

`bool FractionConfig::valid = true`

Indicates whether the row loaded from configuration passed validation.

Definition at line 34 of file [FractionConfig.hpp](#).

Referenced by [Army::Army\(\)](#), and [ConfigLoader::LoadFromFile\(\)](#).

The documentation for this struct was generated from the following file:

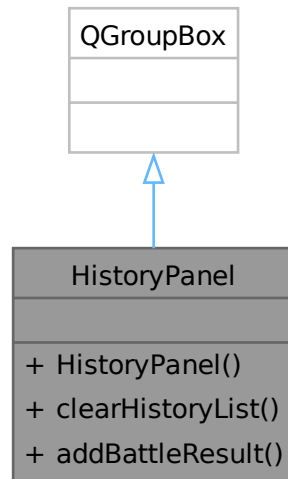
- `include/config/FractionConfig.hpp`

6.14 HistoryPanel Class Reference

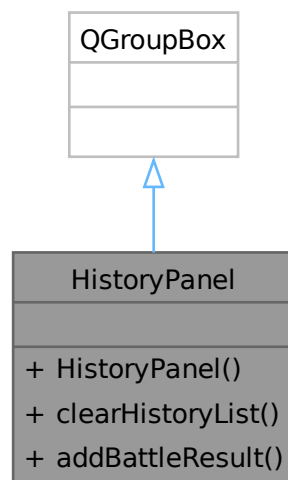
GUI panel listing completed and skipped battles.

```
#include <HistoryPanel.hpp>
```

Inheritance diagram for HistoryPanel:



Collaboration diagram for HistoryPanel:



Public Member Functions

- [HistoryPanel](#) (QWidget *parent=nullptr)
Creates the history list widget.
- void [clearHistoryList](#) ()
Removes all entries from the history list.
- void [addBattleResult](#) (const QString &result)
Appends one battle or campaign result line.

6.14.1 Detailed Description

GUI panel listing completed and skipped battles.

Definition at line 14 of file [HistoryPanel.hpp](#).

6.14.2 Constructor & Destructor Documentation

6.14.2.1 HistoryPanel()

```
HistoryPanel::HistoryPanel (  
    QWidget * parent = nullptr)
```

Creates the history list widget.

Definition at line 10 of file [HistoryPanel.cpp](#).

6.14.3 Member Function Documentation

6.14.3.1 addBattleResult()

```
void HistoryPanel::addBattleResult (  
    const QString & result)
```

Appends one battle or campaign result line.

Definition at line 31 of file [HistoryPanel.cpp](#).

6.14.3.2 clearHistoryList()

```
void HistoryPanel::clearHistoryList ()
```

Removes all entries from the history list.

Definition at line 26 of file [HistoryPanel.cpp](#).

The documentation for this class was generated from the following files:

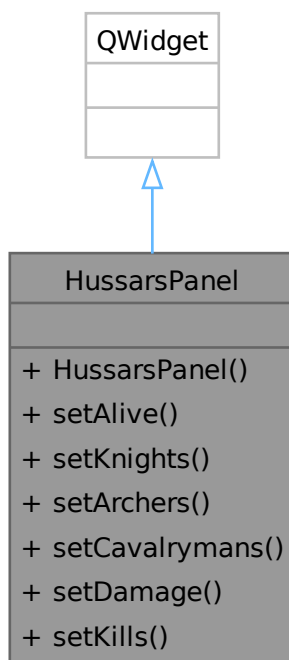
- include/gui/HistoryPanel.hpp
- src/gui/HistoryPanel.cpp

6.15 HussarsPanel Class Reference

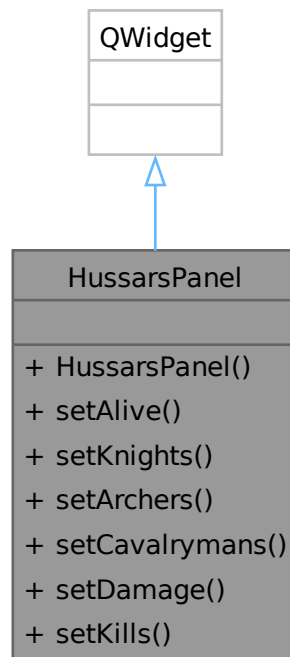
GUI panel displaying current Hussars army statistics.

```
#include <HussarsPanel.hpp>
```

Inheritance diagram for HussarsPanel:



Collaboration diagram for HussarsPanel:



Public Member Functions

- [HussarsPanel](#) (`QWidget *parent=nullptr`)
Creates labels and image for the Hussars army panel.
- void [setAlive](#) (`int alive`)
Displays count of all living Hussars warriors.
- void [setKnights](#) (`int knights`)
Displays count of living knights.
- void [setArchers](#) (`int archers`)
Displays count of living archers.
- void [setCavalrymans](#) (`int cavalrymans`)
Displays count of living cavalrymen.
- void [setDamage](#) (`double damage`)
Displays total damage dealt by Hussars.
- void [setKills](#) (`int kills`)
Displays total kills made by Hussars.

6.15.1 Detailed Description

GUI panel displaying current Hussars army statistics.

Definition at line 13 of file [HussarsPanel.hpp](#).

6.15.2 Constructor & Destructor Documentation

6.15.2.1 HussarsPanel()

```
HussarsPanel::HussarsPanel (  
    QWidget * parent = nullptr)
```

Creates labels and image for the Hussars army panel.

Definition at line 12 of file [HussarsPanel.cpp](#).

6.15.3 Member Function Documentation

6.15.3.1 setAlive()

```
void HussarsPanel::setAlive (  
    int alive)
```

Displays count of all living Hussars warriors.

Definition at line 61 of file [HussarsPanel.cpp](#).

6.15.3.2 setArchers()

```
void HussarsPanel::setArchers (  
    int archers)
```

Displays count of living archers.

Definition at line 71 of file [HussarsPanel.cpp](#).

6.15.3.3 setCavalrymans()

```
void HussarsPanel::setCavalrymans (  
    int cavalrymans)
```

Displays count of living cavalymen.

Definition at line 75 of file [HussarsPanel.cpp](#).

6.15.3.4 setDamage()

```
void HussarsPanel::setDamage (  
    double damage)
```

Displays total damage dealt by Hussars.

Definition at line 79 of file [HussarsPanel.cpp](#).

6.15.3.5 setKills()

```
void HussarsPanel::setKills (  
    int kills)
```

Displays total kills made by Hussars.

Definition at line 85 of file [HussarsPanel.cpp](#).

6.15.3.6 setKnights()

```
void HussarsPanel::setKnights (  
    int knights)
```

Displays count of living knights.

Definition at line 67 of file [HussarsPanel.cpp](#).

The documentation for this class was generated from the following files:

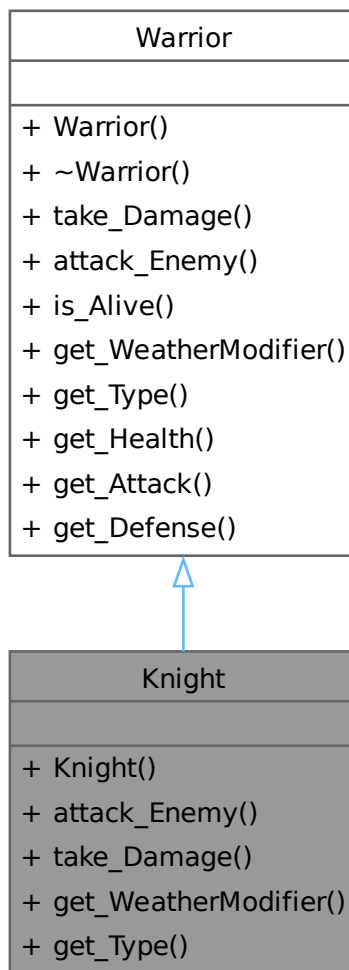
- include/gui/HussarsPanel.hpp
- src/gui/HussarsPanel.cpp

6.16 Knight Class Reference

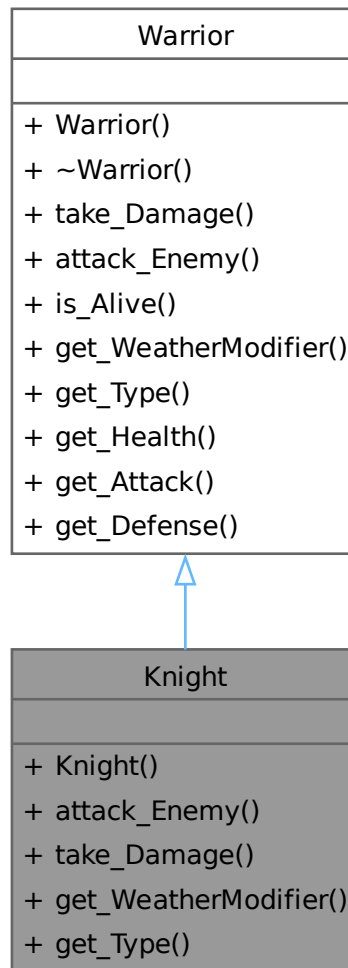
Heavy infantry unit with attack and defense bonuses.

```
#include <Knight.hpp>
```

Inheritance diagram for Knight:



Collaboration diagram for Knight:



Public Member Functions

- `Knight` (const `UnitStats` &stats)
Creates a knight from base statistics.
- void `attack_Enemy` (`Warrior` &enemy, double moraleModifier, WeatherType weather) override
Performs a knight attack against another warrior.
- void `take_Damage` (double damage) override
Applies incoming damage using knight defense behavior.
- double `get_WeatherModifier` (WeatherType weather) const override
Returns knight-specific weather modifier.
- std::string `get_Type` () const override
Returns "Knight".

Public Member Functions inherited from [Warrior](#)

- [Warrior](#) ([UnitStats](#))
Creates a warrior from base unit statistics.
- virtual `~Warrior ()`=default
Enables safe destruction through a base pointer.
- bool [is_Alive](#) () const
Checks whether the warrior can still fight.
- int [get_Health](#) () const
Returns current health.
- int [get_Attack](#) () const
Returns base attack value.
- int [get_Defense](#) () const
Returns base defense value.

6.16.1 Detailed Description

Heavy infantry unit with attack and defense bonuses.

Definition at line 13 of file [Knight.hpp](#).

6.16.2 Constructor & Destructor Documentation

6.16.2.1 [Knight\(\)](#)

```
Knight::Knight (  
    const UnitStats & stats) [explicit]
```

Creates a knight from base statistics.

Definition at line 10 of file [Knight.cpp](#).

References [Warrior::Warrior\(\)](#).

6.16.3 Member Function Documentation

6.16.3.1 [attack_Enemy\(\)](#)

```
void Knight::attack_Enemy (  
    Warrior & enemy,  
    double moraleModifier,  
    WeatherType weather) [override], [virtual]
```

Performs a knight attack against another warrior.

Implements [Warrior](#).

Definition at line 32 of file [Knight.cpp](#).

References [Warrior::get_Attack\(\)](#), [Warrior::get_Defense\(\)](#), [get_WeatherModifier\(\)](#), [Warrior::take_Damage\(\)](#), and [Warrior::Warrior\(\)](#).

6.16.3.2 `get_Type()`

```
std::string Knight::get_Type () const [inline], [override], [virtual]
```

Returns "Knight".

Implements [Warrior](#).

Definition at line 33 of file [Knight.hpp](#).

6.16.3.3 `get_WeatherModifier()`

```
double Knight::get_WeatherModifier (  
    WeatherType weather) const [override], [virtual]
```

Returns knight-specific weather modifier.

Implements [Warrior](#).

Definition at line 16 of file [Knight.cpp](#).

Referenced by [attack_Enemy\(\)](#).

6.16.3.4 `take_Damage()`

```
void Knight::take_Damage (  
    double damage) [override], [virtual]
```

Applies incoming damage using knight defense behavior.

Reimplemented from [Warrior](#).

Definition at line 41 of file [Knight.cpp](#).

References [Warrior::take_Damage\(\)](#).

The documentation for this class was generated from the following files:

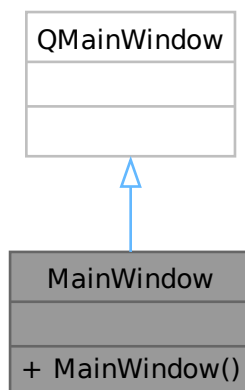
- [include/warriors/Knight.hpp](#)
- [src/warriors/Knight.cpp](#)

6.17 MainWindow Class Reference

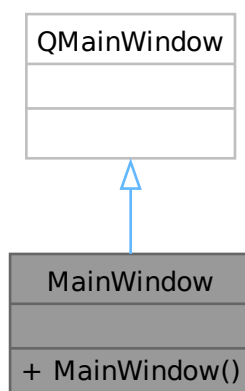
Main Qt window coordinating configuration, simulation and reports.

```
#include <MainWindow.hpp>
```

Inheritance diagram for MainWindow:



Collaboration diagram for MainWindow:



Public Member Functions

- [MainWindow](#) (QWidget *parent=nullptr)
Builds the main application window and connects UI signals.

6.17.1 Detailed Description

Main Qt window coordinating configuration, simulation and reports.

Definition at line 22 of file [MainWindow.hpp](#).

6.17.2 Constructor & Destructor Documentation

6.17.2.1 MainWindow()

```
MainWindow::MainWindow (  
    QWidget * parent = nullptr)
```

Builds the main application window and connects UI signals.

Definition at line 27 of file [MainWindow.cpp](#).

The documentation for this class was generated from the following files:

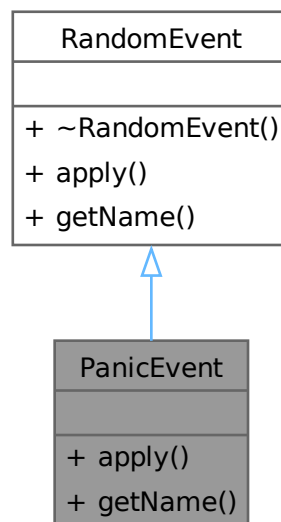
- [include/gui/MainWindow.hpp](#)
- [src/gui/MainWindow.cpp](#)

6.18 PanicEvent Class Reference

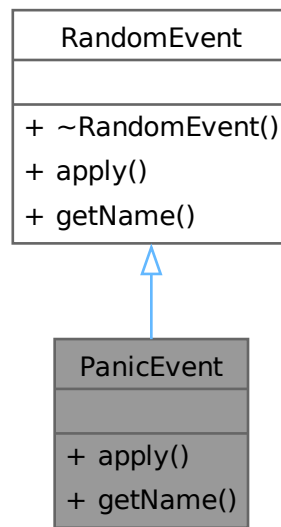
[Random](#) event that decreases army morale.

```
#include <PanicEvent.hpp>
```

Inheritance diagram for PanicEvent:



Collaboration diagram for PanicEvent:



Public Member Functions

- void [apply](#) ([Army](#) &army) override
Decreases morale of the affected army.
- std::string [getName](#) () const override
Returns the display name of the event.

Public Member Functions inherited from [RandomEvent](#)

- virtual [~RandomEvent](#) ()=default
Allows deleting derived event objects through a base pointer.

6.18.1 Detailed Description

[Random](#) event that decreases army morale.

Definition at line 12 of file [PanicEvent.hpp](#).

6.18.2 Member Function Documentation

6.18.2.1 [apply\(\)](#)

```
void PanicEvent::apply (
    Army & army) [override], [virtual]
```

Decreases morale of the affected army.

Implements [RandomEvent](#).

Definition at line 7 of file [PanicEvent.cpp](#).

References [Army::decrease_Morale\(\)](#).

6.18.2.2 getName()

```
std::string PanicEvent::getName () const [override], [virtual]
```

Returns the display name of the event.

Implements [RandomEvent](#).

Definition at line 11 of file [PanicEvent.cpp](#).

The documentation for this class was generated from the following files:

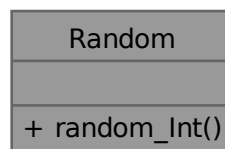
- include/events/PanicEvent.hpp
- src/events/PanicEvent.cpp

6.19 Random Class Reference

Utility wrapper for generating random integer values.

```
#include <Random.hpp>
```

Collaboration diagram for Random:



Static Public Member Functions

- static int [random_Int](#) (int min, int max)
Returns a random integer from the inclusive range [min, max].

6.19.1 Detailed Description

Utility wrapper for generating random integer values.

Definition at line 11 of file [Random.hpp](#).

6.19.2 Member Function Documentation

6.19.2.1 random_Int()

```
int Random::random_Int (
    int min,
    int max) [static]
```

Returns a random integer from the inclusive range [min, max].

Definition at line 11 of file [Random.cpp](#).

Referenced by [Army::add_Reinforcements\(\)](#), [AlliedHelp::apply\(\)](#), [Archer::attack_Enemy\(\)](#), [Battle::do_Round\(\)](#), [Army::get_RandomAliveWarrior\(\)](#), and [Weather::Weather\(\)](#).

The documentation for this class was generated from the following files:

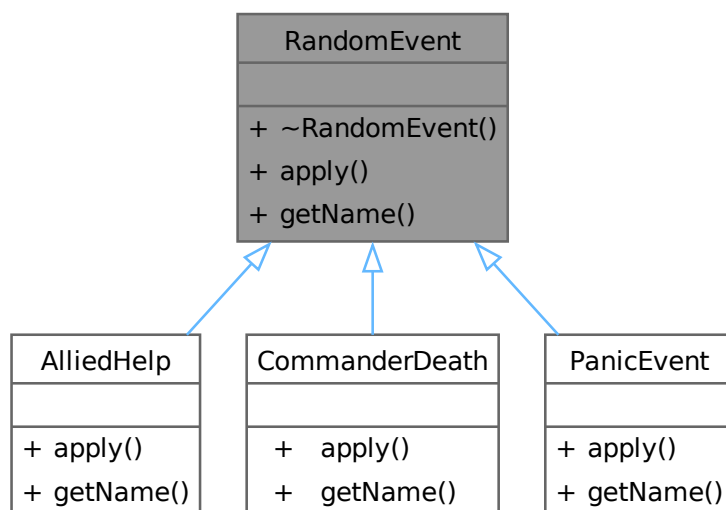
- include/utils/Random.hpp
- src/utils/Random.cpp

6.20 RandomEvent Class Reference

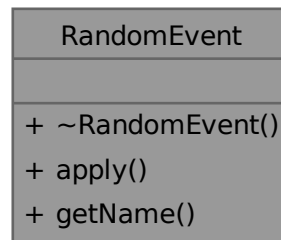
Interface for random events that can affect an army during battle.

```
#include <RandomEvent.hpp>
```

Inheritance diagram for RandomEvent:



Collaboration diagram for RandomEvent:



Public Member Functions

- virtual ~**RandomEvent** ()=default
Allows deleting derived event objects through a base pointer.
- virtual void **apply** ([Army](#) &army)=0
Applies the event effect to the selected army.
- virtual std::string **getName** () const =0
Returns event name displayed in the current-event label.

6.20.1 Detailed Description

Interface for random events that can affect an army during battle.

Definition at line 14 of file [RandomEvent.hpp](#).

6.20.2 Member Function Documentation

6.20.2.1 **apply()**

```
virtual void RandomEvent::apply (
    Army & army) [pure virtual]
```

Applies the event effect to the selected army.

Parameters

<i>army</i>	Army affected by the event.
-------------	---

Implemented in [AlliedHelp](#), [CommanderDeath](#), and [PanicEvent](#).

6.20.2.2 getName()

```
virtual std::string RandomEvent::getName () const [pure virtual]
```

Returns event name displayed in the current-event label.

Implemented in [AlliedHelp](#), [CommanderDeath](#), and [PanicEvent](#).

The documentation for this class was generated from the following file:

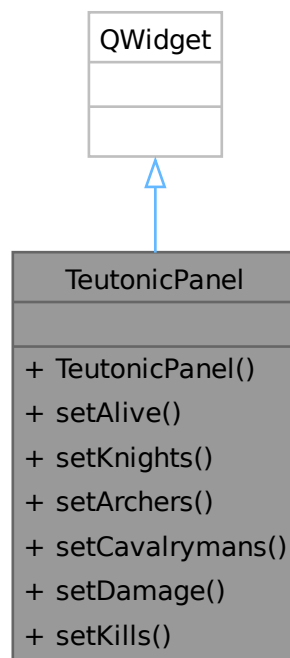
- include/events/RandomEvent.hpp

6.21 TeutonicPanel Class Reference

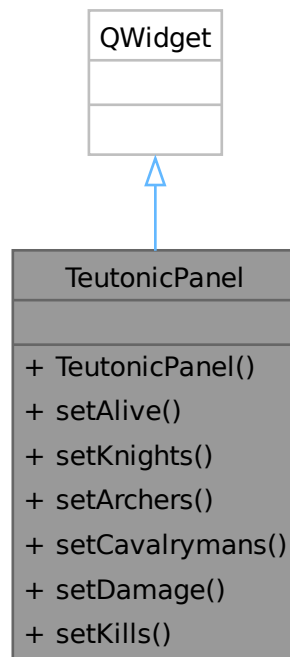
GUI panel displaying current Teutonic army statistics.

```
#include <TeutonicPanel.hpp>
```

Inheritance diagram for TeutonicPanel:



Collaboration diagram for TeutonicPanel:



Public Member Functions

- [TeutonicPanel](#) (`QWidget *parent=nullptr`)
Creates labels and image for the Teutonic army panel.
- void [setAlive](#) (`int alive`)
Displays count of all living Teutonic warriors.
- void [setKnights](#) (`int knights`)
Displays count of living knights.
- void [setArchers](#) (`int archers`)
Displays count of living archers.
- void [setCavalrymans](#) (`int cavalrymans`)
Displays count of living cavalrymen.
- void [setDamage](#) (`double damage`)
Displays total damage dealt by Teutonic army.
- void [setKills](#) (`int kills`)
Displays total kills made by Teutonic army.

6.21.1 Detailed Description

GUI panel displaying current Teutonic army statistics.

Definition at line 15 of file [TeutonicPanel.hpp](#).

6.21.2 Constructor & Destructor Documentation

6.21.2.1 TeutonicPanel()

```
TeutonicPanel::TeutonicPanel (  
    QWidget * parent = nullptr)
```

Creates labels and image for the Teutonic army panel.

Definition at line 11 of file [TeutonicPanel.cpp](#).

6.21.3 Member Function Documentation

6.21.3.1 setAlive()

```
void TeutonicPanel::setAlive (  
    int alive)
```

Displays count of all living Teutonic warriors.

Definition at line 59 of file [TeutonicPanel.cpp](#).

6.21.3.2 setArchers()

```
void TeutonicPanel::setArchers (  
    int archers)
```

Displays count of living archers.

Definition at line 69 of file [TeutonicPanel.cpp](#).

6.21.3.3 setCavalrymans()

```
void TeutonicPanel::setCavalrymans (  
    int cavalrymans)
```

Displays count of living cavalymen.

Definition at line 73 of file [TeutonicPanel.cpp](#).

6.21.3.4 setDamage()

```
void TeutonicPanel::setDamage (  
    double damage)
```

Displays total damage dealt by Teutonic army.

Definition at line 77 of file [TeutonicPanel.cpp](#).

6.21.3.5 setKills()

```
void TeutonicPanel::setKills (  
    int kills)
```

Displays total kills made by Teutonic army.

Definition at line 83 of file [TeutonicPanel.cpp](#).

6.21.3.6 setKnights()

```
void TeutonicPanel::setKnights (  
    int knights)
```

Displays count of living knights.

Definition at line 65 of file [TeutonicPanel.cpp](#).

The documentation for this class was generated from the following files:

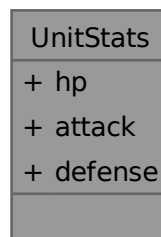
- [include/gui/TeutonicPanel.hpp](#)
- [src/gui/TeutonicPanel.cpp](#)

6.22 UnitStats Struct Reference

Holds base statistics shared by every warrior type.

```
#include <UnitStats.hpp>
```

Collaboration diagram for UnitStats:



Public Attributes

- int [hp](#)
Initial hit points.
- int [attack](#)
Base attack value used when calculating damage.
- int [defense](#)
Base defense value used when reducing incoming damage.

6.22.1 Detailed Description

Holds base statistics shared by every warrior type.

Definition at line 11 of file [UnitStats.hpp](#).

6.22.2 Member Data Documentation

6.22.2.1 attack

```
int UnitStats::attack
```

Base attack value used when calculating damage.

Definition at line 15 of file [UnitStats.hpp](#).

Referenced by [ConfigLoader::LoadFromFile\(\)](#).

6.22.2.2 defense

```
int UnitStats::defense
```

Base defense value used when reducing incoming damage.

Definition at line 17 of file [UnitStats.hpp](#).

Referenced by [ConfigLoader::LoadFromFile\(\)](#).

6.22.2.3 hp

```
int UnitStats::hp
```

Initial hit points.

Definition at line 13 of file [UnitStats.hpp](#).

Referenced by [ConfigLoader::LoadFromFile\(\)](#).

The documentation for this struct was generated from the following file:

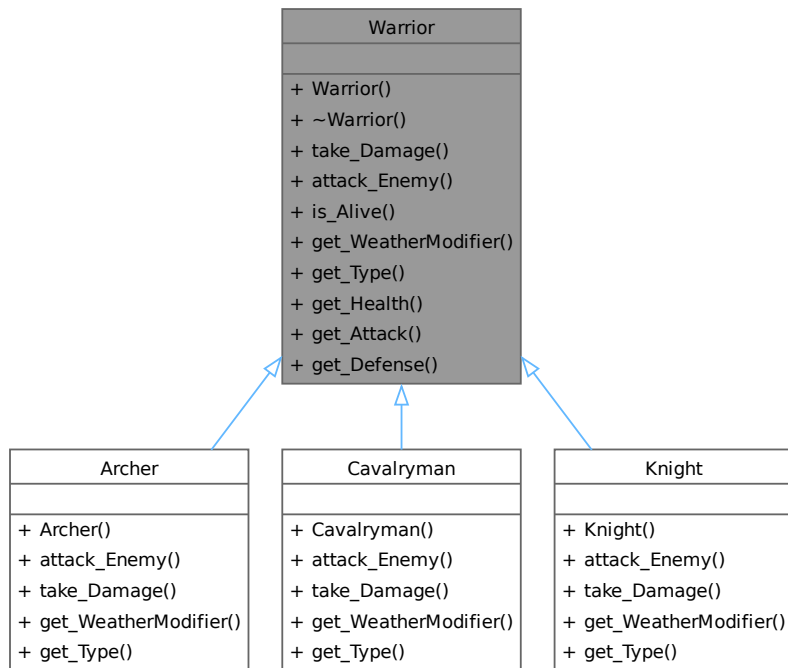
- [include/core/UnitStats.hpp](#)

6.23 Warrior Class Reference

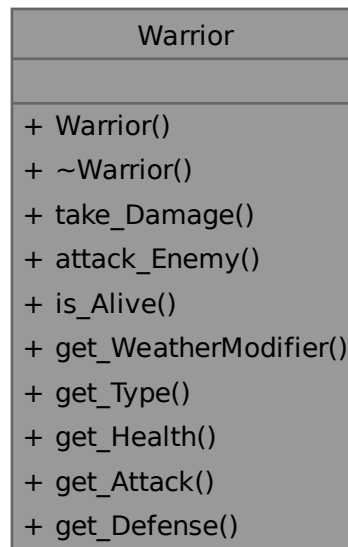
Abstract base class for all battle units.

```
#include <Warrior.hpp>
```

Inheritance diagram for Warrior:



Collaboration diagram for Warrior:



Public Member Functions

- [Warrior](#) ([UnitStats](#))
Creates a warrior from base unit statistics.
- virtual `~Warrior ()`=default
Enables safe destruction through a base pointer.
- virtual void [take_Damage](#) (double damage)
Applies damage and marks the warrior as dead when health reaches zero.
- virtual void [attack_Enemy](#) ([Warrior](#) &enemy, double moraleModifier, WeatherType weather)=0
Attacks another warrior.
- bool [is_Alive](#) () const
Checks whether the warrior can still fight.
- virtual double [get_WeatherModifier](#) (WeatherType weather) const =0
Returns unit-specific modifier for the given weather.
- virtual std::string [get_Type](#) () const =0
Returns a display name of the concrete warrior type.
- int [get_Health](#) () const
Returns current health.
- int [get_Attack](#) () const
Returns base attack value.
- int [get_Defense](#) () const
Returns base defense value.

6.23.1 Detailed Description

Abstract base class for all battle units.

Definition at line 16 of file [Warrior.hpp](#).

6.23.2 Constructor & Destructor Documentation

6.23.2.1 Warrior()

```
Warrior::Warrior (  
    UnitStats stats) [explicit]
```

Creates a warrior from base unit statistics.

Parameters

<i>stats</i>	Initial health, attack and defense values.
--------------	--

Definition at line 11 of file [Warrior.cpp](#).

Referenced by [Archer::Archer\(\)](#), [Archer::attack_Energy\(\)](#), [Cavalryman::attack_Energy\(\)](#), [Knight::attack_Energy\(\)](#), [attack_Energy\(\)](#), [Cavalryman::Cavalryman\(\)](#), and [Knight::Knight\(\)](#).

6.23.3 Member Function Documentation

6.23.3.1 attack_Energy()

```
virtual void Warrior::attack_Energy (  
    Warrior & enemy,  
    double moraleModifier,  
    WeatherType weather) [pure virtual]
```

Attacks another warrior.

Parameters

<i>enemy</i>	Target warrior.
<i>moraleModifier</i>	Army morale modifier used in damage calculations.
<i>weather</i>	Current weather affecting unit effectiveness.

Implemented in [Archer](#), [Cavalryman](#), and [Knight](#).

References [Warrior\(\)](#).

Referenced by [Battle::do_Round\(\)](#).

6.23.3.2 get_Attack()

```
int Warrior::get_Attack () const
```

Returns base attack value.

Definition at line 30 of file [Warrior.cpp](#).

Referenced by [Archer::attack_Energy\(\)](#), [Cavalryman::attack_Energy\(\)](#), and [Knight::attack_Energy\(\)](#).

6.23.3.3 `get_Defense()`

```
int Warrior::get_Defense () const
```

Returns base defense value.

Definition at line 34 of file [Warrior.cpp](#).

Referenced by [Archer::attack_Energy\(\)](#), [Cavalryman::attack_Energy\(\)](#), and [Knight::attack_Energy\(\)](#).

6.23.3.4 `get_Health()`

```
int Warrior::get_Health () const
```

Returns current health.

Definition at line 26 of file [Warrior.cpp](#).

Referenced by [Battle::do_Round\(\)](#).

6.23.3.5 `get_Type()`

```
virtual std::string Warrior::get_Type () const [pure virtual]
```

Returns a display name of the concrete warrior type.

Implemented in [Archer](#), [Cavalryman](#), and [Knight](#).

Referenced by [Battle::do_Round\(\)](#).

6.23.3.6 `get_WeatherModifier()`

```
virtual double Warrior::get_WeatherModifier (  
    WeatherType weather) const [pure virtual]
```

Returns unit-specific modifier for the given weather.

Implemented in [Archer](#), [Cavalryman](#), and [Knight](#).

6.23.3.7 `is_Alive()`

```
bool Warrior::is_Alive () const
```

Checks whether the warrior can still fight.

Definition at line 15 of file [Warrior.cpp](#).

Referenced by [Battle::do_Round\(\)](#).

6.23.3.8 take_Damage()

```
void Warrior::take_Damage (
    double damage) [virtual]
```

Applies damage and marks the warrior as dead when health reaches zero.

Parameters

<i>damage</i>	Raw damage value after attacker-side calculations.
---------------	--

Reimplemented in [Archer](#), [Cavalryman](#), and [Knight](#).

Definition at line 19 of file [Warrior.cpp](#).

Referenced by [Archer::attack_Enemy\(\)](#), [Cavalryman::attack_Enemy\(\)](#), [Knight::attack_Enemy\(\)](#), [Archer::take_Damage\(\)](#), [Cavalryman::take_Damage\(\)](#), and [Knight::take_Damage\(\)](#).

The documentation for this class was generated from the following files:

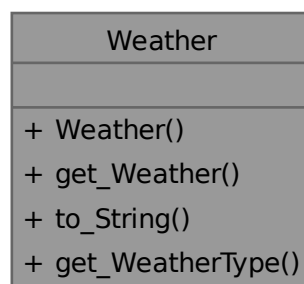
- include/warriors/Warrior.hpp
- src/warriors/Warrior.cpp

6.24 Weather Class Reference

Represents weather selected for one battle.

```
#include <Weather.hpp>
```

Collaboration diagram for Weather:



Public Member Functions

- [Weather](#) ()
Creates a weather object with a randomly selected weather type.
- WeatherType [get_Weather](#) () const
Returns the selected weather type.
- std::string [to_String](#) () const
Converts the selected weather type to text used in GUI and reports.
- WeatherType [get_WeatherType](#) () const
Returns the selected weather type.

6.24.1 Detailed Description

Represents weather selected for one battle.

Definition at line 15 of file [Weather.hpp](#).

6.24.2 Constructor & Destructor Documentation

6.24.2.1 Weather()

```
Weather::Weather ()
```

Creates a weather object with a randomly selected weather type.

Definition at line 9 of file [Weather.cpp](#).

References [Random::random_Int\(\)](#).

6.24.3 Member Function Documentation

6.24.3.1 get_WeatherType()

```
WeatherType Weather::get_WeatherType () const
```

Returns the selected weather type.

Definition at line 31 of file [Weather.cpp](#).

6.24.3.2 to_String()

```
std::string Weather::to_String () const
```

Converts the selected weather type to text used in GUI and reports.

Definition at line 37 of file [Weather.cpp](#).

The documentation for this class was generated from the following files:

- include/events/Weather.hpp
- src/events/Weather.cpp

Chapter 7

File Documentation

7.1 ConfigLoader.hpp

```
00001 //
00002 // Created by bart on 11.05.2026.
00003
00004 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_CONFIGLOADER_HPP
00005 #define SIMULATION_BATTLE_OF_GRUNWALD_CONFIGLOADER_HPP
00006
00007 #include <string>
00008 #include <vector>
00009 #include "FractionConfig.hpp"
00010
00014 class ConfigLoader{
00015     public:
00023         static std::vector<FractionConfig> LoadFromFile(const std::string& filename, bool
printErrors);
00024 };
00025
00026 #endif //SIMULATION_BATTLE_OF_GRUNWALD_CONFIGLOADER_HPP
```

7.2 FractionConfig.hpp

```
00001 //
00002 // Created by bart on 11.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_BATTLECONFIG_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_BATTLECONFIG_HPP
00007
00008 #include <core/UnitStats.hpp>
00009 #include <string>
00010
00014 struct FractionConfig {
00016     std::string fraction;
00017
00019     UnitStats knight_stats;
00021     int knight_count;
00022
00024     UnitStats archer_stats;
00026     int archer_count;
00027
00029     UnitStats cavalry_stats;
00031     int cavalry_count;
00032
00034     bool valid = true;
00035 };
00036
00037 #endif //SIMULATION_BATTLE_OF_GRUNWALD_BATTLECONFIG_HPP
```

7.3 Army.hpp

```

00001 //
00002 // Created by bart on 13.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_ARMY_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_ARMY_HPP
00007
00008
00009 #include <string>
00010 #include <vector>
00011 #include <memory>
00012
00013 #include <warriors/Warrior.hpp>
00014 #include <config/FractionConfig.hpp>
00015
00016
00023 class Army {
00024 private:
00025
00026     std::string name;
00027     std::vector<std::shared_ptr<Warrior>> warriors;
00028
00029     UnitStats knightStats;
00030     UnitStats archerStats;
00031     UnitStats cavalryStats;
00032
00033     int attackBonus;
00034     int defenseBonus;
00035     int moraleLevel;
00036
00037 public:
00038
00039     Army(const FractionConfig& config);
00040
00041     std::size_t get_ArmySize() const;
00042
00043     const std::vector<std::shared_ptr<Warrior>>& get_Warriors() const;
00044
00045     int count_AliveWarriors() const;
00046
00047     int count_AliveKnights() const;
00048     int count_AliveArchers() const;
00049     int count_AliveCavalry() const;
00050
00051     Warrior& get_RandomAliveWarrior();
00052
00053     int get_moraleLevel() const;
00054     double get_MoraleModifier() const;
00055     const std::string& get_Name() const;
00056
00057     void add_Reinforcements(int amount);
00058     void increase_Morale(int amount);
00059     void decrease_Morale(int amount);
00060
00061 };
00062
00063 #endif //SIMULATION_BATTLE_OF_GRUNWALD_ARMY_HPP

```

7.4 Battle.hpp

```

00001 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_BATTLE_HPP
00002 #define SIMULATION_BATTLE_OF_GRUNWALD_BATTLE_HPP
00003
00004 #include <string>
00005 #include <vector>
00006
00007 #include <core/Army.hpp>
00008 #include <events/Weather.hpp>
00009 #include "BattleStats.hpp"
00010 #include "observer/BattleObserver.hpp"
00011 #include <events/RandomEvent.hpp>
00012
00013
00020 class Battle {
00021 private:
00022
00023     Army Hussars;
00024     Army Teutonic;
00025

```



```

00026     int round{};
00027     std::string winner;
00028
00029     Weather weather;
00030
00031     BattleStats stats;
00032
00033     std::vector<BattleObserver*> observers;
00034
00035     std::string currentEvent;
00036
00037     std::vector<std::unique_ptr<RandomEvent>> randomEvents;
00038     int eventsTriggered{};
00039     const int maxEvents{};
00040     void triggerRandomEvent();
00041
00042 public:
00043
00047     Battle(Army& HussarsArmy, Army& TeutonicArmy);
00048
00052     void start_Battle();
00053
00057     void do_Round();
00061     bool is_finished() const;
00062
00066     void finishBattle();
00067
00071     void attach(BattleObserver* observer);
00075     void detach(BattleObserver* observer);
00079     void notify();
00080
00082     int get_Rounds() const;
00083
00085     std::string get_Winner() const;
00086
00088     const Army& get_HussarsArmy() const;
00090     const Army& get_TeutonicArmy() const;
00091
00093     std::string get_Weather() const;
00094
00096     const BattleStats& get_Stats() const;
00097
00099     const std::string& get_CurrentEvent() const;
00100
00101 };
00102
00103 #endif //SIMULATION_BATTLE_OF_GRUNWALD_BATTLE_HPP

```

7.5 BattleStats.hpp

```

00001 //
00002 // Created by bart on 30.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_BATTLESTATS_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_BATTLESTATS_HPP
00007
00011 class BattleStats {
00012 private:
00013     double HussarsDamage_Dealt{};
00014     double TeutonicDamage_Dealt{};
00015
00016     int HussarsKills{};
00017     int TeutonicKills{};
00018
00019 public:
00021     void add_HussarsDamage(double damage);
00023     void add_TeutonicDamage(double damage);
00024
00026     void add_HussarsKill();
00028     void add_TeutonicKill();
00029
00031     double get_HussarsDamage_Dealt() const;
00033     double get_TeutonicDamage_Dealt() const;
00034
00036     int get_HussarsKills() const;
00038     int get_TeutonicKills() const;
00039
00040 };
00041
00042 #endif //SIMULATION_BATTLE_OF_GRUNWALD_BATTLESTATS_HPP

```

7.6 UnitStats.hpp

```
00001 //
00002 // Created by bart on 11.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_UNITSTATS_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_UNITSTATS_HPP
00007
00011 struct UnitStats {
00013     int hp;
00015     int attack;
00017     int defense;
00018 };
00019
00020 #endif //SIMULATION_BATTLE_OF_GRUNWALD_UNITSTATS_HPP
```

7.7 AlliedHelp.hpp

```
00001 //
00002 // Created by bart on 1.06.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_ALLIEDHELP_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_ALLIEDHELP_HPP
00007 #include "RandomEvent.hpp"
00008
00012 class AlliedHelp : public RandomEvent {
00013 public:
00015     void apply(Army& army) override;
00017     std::string getName() const override;
00018 };
00019
00020 #endif //SIMULATION_BATTLE_OF_GRUNWALD_ALLIEDHELP_HPP
```

7.8 CommanderDeath.hpp

```
00001 //
00002 // Created by bart on 1.06.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_COMMANDERDEATH_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_COMMANDERDEATH_HPP
00007 #include "RandomEvent.hpp"
00008
00012 class CommanderDeath : public RandomEvent {
00013 public:
00015     void apply(Army& army) override;
00017     std::string getName() const override;
00018 };
00019
00020 #endif //SIMULATION_BATTLE_OF_GRUNWALD_COMMANDERDEATH_HPP
```

7.9 PanicEvent.hpp

```
00001 //
00002 // Created by bart on 1.06.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_PANICEVENT_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_PANICEVENT_HPP
00007 #include "RandomEvent.hpp"
00008
00012 class PanicEvent : public RandomEvent {
00013 public:
00015     void apply(Army& army) override;
00017     std::string getName() const override;
00018 };
00019
00020 #endif //SIMULATION_BATTLE_OF_GRUNWALD_PANICEVENT_HPP
```

7.10 RandomEvent.hpp

```

00001 //
00002 // Created by bartl on 31.05.2026.
00003 //
00004
00005
00006 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_RANDOMEVENT_HPP
00007 #define SIMULATION_BATTLE_OF_GRUNWALD_RANDOMEVENT_HPP
00008
00009 #include <core/Army.hpp>
00010
00014 class RandomEvent
00015 {
00016 public:
00018     virtual ~RandomEvent() = default;
00019
00024     virtual void apply(Army& army) = 0;
00028     virtual std::string getName() const = 0;
00029 };
00030
00031 #endif //SIMULATION_BATTLE_OF_GRUNWALD_RANDOMEVENT_HPP

```

7.11 Weather.hpp

```

00001 //
00002 // Created by bartl on 15.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_WEATHER_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_WEATHER_HPP
00007
00008 #include <string>
00009 #include <events/WeatherType.hpp>
00010
00011
00015 class Weather {
00016 private:
00017
00018     WeatherType weatherType;
00019
00020 public:
00021
00025     Weather();
00026
00030     WeatherType get_Weather() const;
00031
00035     std::string to_String() const;
00036
00040     WeatherType get_WeatherType() const;
00041 };
00042
00043 #endif //SIMULATION_BATTLE_OF_GRUNWALD_WEATHER_HPP

```

7.12 WeatherType.hpp

```

00001 //
00002 // Created by bart on 28.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_WEATHERTYPE_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_WEATHERTYPE_HPP
00007
00011 enum class WeatherType {
00012     SUNNY,
00013     RAINY,
00014     FOGGY,
00015     SNOWY
00016 };
00017
00018 #endif //SIMULATION_BATTLE_OF_GRUNWALD_WEATHERTYPE_HPP

```

7.13 BattlePanel.hpp

```

00001 //

```

```

00002 // Created by bart on 31.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_BATTLEPANEL_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_BATTLEPANEL_HPP
00007
00008 #include "core/Battle.hpp"
00009 #include "observer/BattleObserver.hpp"
00010
00011 #include <QWidget>
00012 #include <QLabel>
00013
00014
00015 class BattlePanel : public QWidget, public BattleObserver {
00016 private:
00017     Battle* battle;
00018
00019     QLabel* battleNumberLabel;
00020     QLabel* roundLabel;
00021     QLabel* weatherLabel;
00022
00023     QLabel* HussarsMoraleLabel;
00024     QLabel* TeutonicMoraleLabel;
00025
00026     QLabel* VictoryChanceLabel;
00027     QLabel* HussarsProbabilityLabel;
00028     QLabel* TeutonicProbabilityLabel;
00029
00030     QLabel* currentEventLabel;
00031
00032     QLabel* winnerLabel;
00033 public:
00034     BattlePanel(QWidget* parent = nullptr);
00035
00036     void update(const Battle& battle) override;
00037
00038     void setBattleNumber(int current, int total);
00039     void setRounds(int rounds);
00040     void setWeather(const QString& weather);
00041     void setHussarsMorale(int morale);
00042     void setTeutonicMorale(int morale);
00043     void setHussarsProbability(double probability);
00044     void setTeutonicProbability(double probability);
00045     void setCurrentEvent(const QString& event);
00046     void setWinner(const QString& winner);
00047 };
00048 #endif //SIMULATION_BATTLE_OF_GRUNWALD_BATTLEPANEL_HPP

```

7.14 ControlPanel.hpp

```

00001 //
00002 // Created by bart on 31.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_CONTROLPANEL_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_CONTROLPANEL_HPP
00007
00008 #include <QWidget>
00009 #include <QPushButton>
00010 #include <QComboBox>
00011
00012 class ControlPanel : public QWidget {
00013 private:
00014     QPushButton* startButton;
00015     QPushButton* loadConfigButton;
00016     QPushButton* exitButton;
00017
00018     QComboBox* speedComboBox;
00019 public:
00020     ControlPanel(QWidget *parent);
00021
00022     QPushButton* get_StartButton() const;
00023     QPushButton* get_LoadButton() const;
00024     QPushButton* get_ExitButton() const;
00025
00026     QComboBox* get_SpeedComboBox() const;
00027 };
00028 #endif //SIMULATION_BATTLE_OF_GRUNWALD_CONTROLPANEL_HPP

```

7.15 HistoryPanel.hpp

```

00001 //
00002 // Created by bart on 31.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_HISTORYPANEL_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_HISTORYPANEL_HPP
00007
00008 #include <QGroupBox>
00009 #include <QListWidget>
00010
00014 class HistoryPanel : public QGroupBox {
00015 private:
00016     QListWidget* historyList;
00017
00018 public:
00022     HistoryPanel(QWidget* parent=nullptr);
00023
00025     void clearHistoryList();
00026
00028     void addBattleResult(const QString& result);
00029 };
00030
00031 #endif //SIMULATION_BATTLE_OF_GRUNWALD_HISTORYPANEL_HPP

```

7.16 HussarsPanel.hpp

```

00001 //
00002 // Created by bart on 1.06.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_HUSSARSPANEL_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_HUSSARSPANEL_HPP
00007 #include <QWidget>
00008 #include <QLabel>
00009
00013 class HussarsPanel : public QWidget {
00014 private:
00015
00016     QLabel* hussarsLabel;
00017     QLabel* hussarsIMG;
00018
00019     QLabel* aliveHussarsLabel;
00020     QLabel* aliveKnightsLabel;
00021     QLabel* aliveArchersLabel;
00022     QLabel* aliveCavalrymansLabel;
00023
00024     QLabel* hussarsDamageLabel;
00025     QLabel* hussarsKillsLabel;
00026
00027
00028 public:
00032     HussarsPanel(QWidget* parent = nullptr);
00033
00035     void setAlive(int alive);
00036
00038     void setKnights(int knights);
00040     void setArchers(int archers);
00042     void setCavalrymans(int cavalrymans);
00043
00045     void setDamage(double damage);
00047     void setKills(int kills);
00048 };
00049
00050 #endif //SIMULATION_BATTLE_OF_GRUNWALD_HUSSARSPANEL_HPP

```

7.17 MainWindow.hpp

```

00001 //
00002 // Created by bart on 31.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_MAINWINDOW_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_MAINWINDOW_HPP
00007
00008 #include <qboxlayout.h>

```

```

00009 #include <QMainWindow>
00010 #include <QTimer>
00011
00012 #include "BattlePanel.hpp"
00013 #include "ControlPanel.hpp"
00014 #include "HistoryPanel.hpp"
00015 #include "HussarsPanel.hpp"
00016 #include "TeutonicPanel.hpp"
00017
00018
00022 class MainWindow : public QMainWindow {
00023 private:
00024
00025     Q_OBJECT
00026
00027     std::vector<FractionConfig> loadedConfig;
00028
00029     size_t currentBattleIndex{};
00030
00031     int hussarsWins{};
00032     int teutonicWins{};
00033
00034     BattlePanel* battlePanel;
00035     HussarsPanel* hussarsPanel;
00036     TeutonicPanel* teutonicPanel;
00037     ControlPanel* controlPanel;
00038     HistoryPanel* historyPanel;
00039
00040     QTimer* battleTimer = nullptr;
00041     Battle* currentBattle = nullptr;
00042
00043     QString configPath = "../assets/configs/";
00044
00046     void startBattle();
00048     void loadConfig();
00050     void startNextBattle();
00051
00052     std::string ReportFilename;
00053
00054 private slots:
00056     void battleStep();
00058     void changeSpeed();
00059
00060 public:
00064     MainWindow(QWidget* parent = nullptr);
00065
00066 };
00067
00068 #endif //SIMULATION_BATTLE_OF_GRUNWALD_MAINWINDOW_HPP

```

7.18 TeutonicPanel.hpp

```

00001 //
00002 // Created by bart on 1.06.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_TEUTONICPANEL_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_TEUTONICPANEL_HPP
00007
00008
00009 #include <QWidget>
00010 #include <QLabel>
00011
00015 class TeutonicPanel : public QWidget {
00016 private:
00017
00018     QLabel* teutonicLabel;
00019     QLabel* teutonicIMG;
00020
00021     QLabel* aliveTeutonicLabel;
00022     QLabel* aliveKnightsLabel;
00023     QLabel* aliveArchersLabel;
00024     QLabel* aliveCavalrymansLabel;
00025
00026     QLabel* teutonicDamageLabel;
00027     QLabel* teutonicKillsLabel;
00028
00029 public:
00033     TeutonicPanel(QWidget* parent = nullptr);
00034
00036     void setAlive(int alive);
00037
00039     void setKnights(int knights);

```

```

00041     void setArchers(int archers);
00043     void setCavalrymans(int cavalrymans);
00044
00046     void setDamage(double damage);
00048     void setKills(int kills);
00049 };
00050
00051 #endif //SIMULATION_BATTLE_OF_GRUNWALD_TEUTONICPANEL_HPP

```

7.19 BattleObserver.hpp

```

00001 //
00002 // Created by bart on 31.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_BATTLEOBSERVER_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_BATTLEOBSERVER_HPP
00007
00008 class Battle;
00009
00013 class BattleObserver {
00014 public:
00016     virtual ~BattleObserver() = default;
00021     virtual void update(const Battle& battle) = 0;
00022 };
00023
00024 #endif //SIMULATION_BATTLE_OF_GRUNWALD_BATTLEOBSERVER_HPP

```

7.20 BattleReport.hpp

```

00001 //
00002 // Created by bart on 25.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_BATTLEREPORT_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_BATTLEREPORT_HPP
00007
00008 #include <string>
00009 #include <core/Battle.hpp>
00010
00014 class BattleReport {
00015 public:
00019     static void save(const Battle& battle, const std::string& filename, int battleNumber);
00023     static void save_skipped(const std::string& filename, int battleNumber, const std::string&
reason);
00024 };
00025
00026
00027
00028 #endif //SIMULATION_BATTLE_OF_GRUNWALD_BATTLEREPORT_HPP

```

7.21 Random.hpp

```

00001 //
00002 // Created by bart on 28.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_RANDOM_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_RANDOM_HPP
00007
00011 class Random {
00012 public:
00016     static int random_Int(int min, int max);
00017 };
00018
00019
00020
00021 #endif //SIMULATION_BATTLE_OF_GRUNWALD_RANDOM_HPP

```

7.22 Archer.hpp

```

00001 //
00002 // Created by bartl on 13.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_ARCHER_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_ARCHER_HPP
00007
00008 #include <warriors/Warrior.hpp>
00009
00013 class Archer : public Warrior {
00014 private:
00015     double atkBonus;
00016     double defBonus;
00017
00018 public:
00022     explicit Archer(const UnitStats& stats);
00023
00025     void attack_Enemy(Warrior& enemy, double moraleModifier, WeatherType weather) override;
00027     void take_Damage(double damage) override;
00029     double get_WeatherModifier(WeatherType weather) const override;
00030
00032     std::string get_Type() const override {
00033         return "Archer";
00034     }
00035
00036 };
00037
00038 #endif //SIMULATION_BATTLE_OF_GRUNWALD_ARCHER_HPP

```

7.23 Cavalryman.hpp

```

00001 //
00002 // Created by bartl on 13.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_CAVALRYMAN_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_CAVALRYMAN_HPP
00007
00008 #include <warriors/Warrior.hpp>
00009
00013 class Cavalryman : public Warrior {
00014 private:
00015     bool firstAttack;
00016     double atkBonus;
00017     double defBonus;
00018
00019 public:
00023     explicit Cavalryman(const UnitStats& stats);
00024
00026     void attack_Enemy(Warrior& enemy, double moraleModifier, WeatherType weather) override;
00028     void take_Damage(double damage) override;
00030     double get_WeatherModifier(WeatherType weather) const override;
00031
00033     std::string get_Type() const override {
00034         return "Cavalryman";
00035     }
00036
00037 };
00038
00039 #endif //SIMULATION_BATTLE_OF_GRUNWALD_CAVALRYMAN_HPP

```

7.24 Knight.hpp

```

00001 //
00002 // Created by bartl on 13.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_KNIGHT_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_KNIGHT_HPP
00007
00008 #include <warriors/Warrior.hpp>
00009
00013 class Knight : public Warrior {
00014 private:
00015     double atkBonus;

```



```

00016     double defBonus;
00017
00018 public:
00022     explicit Knight(const UnitStats& stats);
00023
00024
00026     void attack_Enemy(Warrior& enemy, double moraleModifier, WeatherType weather) override;
00028     void take_Damage(double damage) override;
00030     double get_WeatherModifier(WeatherType weather) const override;
00031
00033     std::string get_Type() const override {
00034         return "Knight";
00035     }
00036
00037 };
00038
00039 #endif //SIMULATION_BATTLE_OF_GRUNWALD_KNIGHT_HPP

```

7.25 Warrior.hpp

```

00001 //
00002 // Created by bartl on 13.05.2026.
00003 //
00004
00005 #ifndef SIMULATION_BATTLE_OF_GRUNWALD_WARRIOR_HPP
00006 #define SIMULATION_BATTLE_OF_GRUNWALD_WARRIOR_HPP
00007
00008
00009 #include <core/UnitStats.hpp>
00010 #include <events/WeatherType.hpp>
00011 #include <string>
00012
00016 class Warrior {
00017 private:
00018
00019     int health;
00020     int attack;
00021     int defense;
00022
00023     bool alive;
00024
00025 public:
00026
00031     explicit Warrior(UnitStats);
00032
00034     virtual ~Warrior() = default;
00035
00040     virtual void take_Damage(double damage);
00041
00049     virtual void attack_Enemy(Warrior& enemy, double moraleModifier, WeatherType weather) = 0;
00050
00054     bool is_Alive() const;
00055
00059     virtual double get_WeatherModifier(WeatherType weather) const=0;
00060
00064     virtual std::string get_Type() const = 0;
00065
00067     int get_Health() const;
00068
00070     int get_Attack() const;
00071
00073     int get_Defense() const;
00074 };
00075
00076
00077 #endif //SIMULATION_BATTLE_OF_GRUNWALD_WARRIOR_HPP

```

7.26 main.cpp

```

00001 #include <iostream>
00002 #include <string>
00003 #include <fstream>
00004 #include <config/ConfigLoader.hpp>
00005 #include <core/Battle.hpp>
00006 #include <reports/BattleReport.hpp>
00007
00008 //Gui
00009 #include <QApplication>

```

```

00010 #include <gui/MainWindow.hpp>
00011
00012 #include "gui/BattlePanel.hpp"
00013
00014
00015 // Battle loop with load config and save result
00016
00017
00018 void battle() {
00019     std::vector<FractionConfig> config =
00020         ConfigLoader::LoadFromFile("../assets/configs/testfile_someinfected.csv", false);
00021
00022     int battleNumber{1};
00023
00024     std::string filename = "../assets/reports/battleReport1.csv";
00025
00026     std::ofstream file(filename);
00027     file<<"Battle, Winner, Rounds, Hussars Alive, Teutonic Alive, Hussars Total Damage, Teutonic Total
00028     Damage, Hussars Kills, Teutonic Kills, Weather, Status, Date\n";
00029     file.close();
00030
00031     for (size_t i = 0; i < config.size()-1; i+=2) {
00032         try {
00033             std::cout<<"======"<<"\n";
00034             std::cout<<"Battle " << battleNumber<<"\n";
00035             std::cout<<config[i].fraction<<" vs " <<config[i+1].fraction<<"\n";
00036             std::cout<<"======"<<"\n";
00037
00038             Army Hussars(config[i]);
00039             Army Teutonic(config[i + 1]);
00040
00041             Battle battle(Hussars, Teutonic);
00042
00043             battle.start_Battle();
00044             std::cout<<"Weather: " << battle.get_Weather() <<"\n";
00045
00046             std::cout<<"Winner: " << battle.get_Winner() <<"\n";
00047
00048             std::cout<<"Rounds: " << battle.get_Rounds() <<"\n";
00049
00050             std::cout<<"Alive
00051             " <<config[i].fraction<<": " << battle.get_HussarsArmy().count_AliveWarriors() <<"\n";
00052             std::cout<<"Alive
00053             " <<config[i+1].fraction<<": " << battle.get_TeutonicArmy().count_AliveWarriors() <<"\n";
00054
00055             std::cout<<"Hussars Total Damage: " << battle.get_Stats().get_HussarsDamage_Dealt() <<"\n";
00056             std::cout<<"Teutonic Total Damage: " << battle.get_Stats().get_TeutonicDamage_Dealt() <<"\n";
00057
00058             std::cout<<"Hussars kills: " << battle.get_Stats().get_HussarsKills() <<"\n";
00059             std::cout<<"Teutonic kills: " << battle.get_Stats().get_TeutonicKills() <<"\n";
00060
00061             BattleReport::save(battle, filename, battleNumber);
00062
00063         }
00064         catch (const std::exception& e) {
00065             std::cout<<"Battle skipped: " << e.what() <<"\n";
00066
00067             BattleReport::save_skipped(filename, battleNumber, e.what());
00068
00069         }
00070
00071         battleNumber++;
00072     }
00073 }
00074
00075 int main(int argc, char *argv[]) {
00076     //battle(
00077     QApplication app(argc, argv);
00078
00079     BattlePanel battlePanel;
00080
00081     MainWindow mainWindow;
00082     mainWindow.show();
00083
00084     return app.exec();
00085 }

```

7.27 ConfigLoader.cpp

```

00001 //

```

```

00002 // Created by bart on 11.05.2026.
00003 //
00004
00005 #include <iostream>
00006 #include <fstream>
00007 #include <sstream>
00008 #include <vector>
00009 #include <string>
00010
00011 #include <config/ConfigLoader.hpp>
00012
00013 bool is_config_valid(const FractionConfig& config);
00014 bool is_integer(const std::vector<std::string>& str);
00015 bool isnot_empty_string(const std::vector<std::string>& str);
00016
00017
00018
00019 // Implementation of loading params from csv file
00020
00021 std::vector<FractionConfig> ConfigLoader::LoadFromFile(const std::string& filename, bool printErrors =
00022 true) {
00023     std::vector<FractionConfig> configs;
00024
00025     std::ifstream file_to_read(filename);
00026     std::string line;
00027
00028     constexpr size_t EXPECTED_FIELDS = 13;
00029
00030     // Handle error while opening file broken/unaccessed
00031     if (!file_to_read) {
00032         if (printErrors) {
00033             std::cerr << "Could not open file " << filename << "\n";
00034         }
00035     }
00036
00037     // Skip header line
00038     std::getline(file_to_read, line);
00039
00040
00041     // Load each Fraction config
00042     while (std::getline(file_to_read, line)) {
00043         if (line.empty() || line[0] == '#') {
00044             continue;
00045         }
00046
00047         FractionConfig config{};
00048         std::vector<std::string> values;
00049
00050         std::stringstream ss(line);
00051         std::string value;
00052
00053         // Get ready line with one fraction values
00054         while (getline(ss, value, ',')) {
00055             values.push_back(value);
00056         }
00057
00058         // Assignment of all corresponding values to fraction and catch errors
00059         if (values.size() == EXPECTED_FIELDS) {
00060             try {
00061                 if (is_integer(values) && isnot_empty_string(values) ) {
00062                     config.fraction = values[0];
00063
00064                     config.knight_stats.hp = std::stoi(values[1]);
00065                     config.knight_stats.attack = std::stoi(values[2]);
00066                     config.knight_stats.defense = std::stoi(values[3]);
00067                     config.knight_count = std::stoi(values[4]);
00068
00069                     config.archer_stats.hp = std::stoi(values[5]);
00070                     config.archer_stats.attack = std::stoi(values[6]);
00071                     config.archer_stats.defense = std::stoi(values[7]);
00072                     config.archer_count = std::stoi(values[8]);
00073
00074                     config.cavalry_stats.hp = std::stoi(values[9]);
00075                     config.cavalry_stats.attack = std::stoi(values[10]);
00076                     config.cavalry_stats.defense = std::stoi(values[11]);
00077                     config.cavalry_count = std::stoi(values[12]);
00078
00079                     if (is_config_valid(config)) {
00080                         configs.push_back(config);
00081                     }
00082                 }
00083                 else {
00084                     config.valid = false;
00085                     configs.push_back(config);
00086                 }
00087             }

```

```

00088             if (printErrors) {
00089                 std::cout<<"Error, all values must be greater than zero" << "\n";
00090             }
00091         }
00092     }
00093     else {
00094         config.valid = false;
00095         configs.push_back(config);
00096         if (printErrors) {
00097             std::cout<<"Error, params of warriors must contain only digits and all values
not be empty" << "\n";
00098         }
00099     }
00100 }
00101 catch (const std::invalid_argument& e) {
00102     config.valid = false;
00103     configs.push_back(config);
00104     if (printErrors) {
00105         std::cerr << "Error, value must be number" << e.what() << "\n";
00106     }
00107 }
00108 catch (const std::out_of_range& e) {
00109     config.valid = false;
00110     configs.push_back(config);
00111     if (printErrors) {
00112         std::cerr << "Error, value out of range" << e.what() << "\n";
00113     }
00114 }
00115 }
00116 else {
00117     config.valid = false;
00118     configs.push_back(config);
00119     if (printErrors) {
00120         std::cerr << "Not enough values!" << filename << "\n";
00121     }
00122 }
00123 }
00124
00125 return configs;
00126 }
00127
00128 // Function for checking is all values is correct
00129 // OPTIONAL SUGGEST add limit for count of types of warriors!
00130 bool is_config_valid(const FractionConfig& config) {
00131
00132     if (config.knight_stats.hp < 0 || config.knight_stats.attack < 0 || config.knight_stats.defense <
00133 0) {
00134         return false;
00135     }
00136     if (config.archer_stats.hp < 0 || config.archer_stats.attack < 0 || config.archer_stats.defense <
00137 0) {
00138         return false;
00139     }
00140     if (config.cavalry_stats.hp < 0 || config.cavalry_stats.attack < 0 || config.cavalry_stats.defense
00141 < 0) {
00142         return false;
00143     }
00144     if (config.knight_count < 0 || config.archer_count < 0 || config.cavalry_count < 0) {
00145         return false;
00146     }
00147     return true;
00148 }
00149 // Validate input is integer
00150 bool is_integer(const std::vector<std::string>& str) {
00151     for (int i = 1; i<13;i++){
00152         for (char c : str[i]) {
00153             if (!std::isdigit(static_cast<unsigned char>(c))) {
00154                 return false;
00155             }
00156         }
00157     }
00158     return true;
00159 }
00160
00161 // Validate is input string not empty
00162 bool isnot_empty_string(const std::vector<std::string>& str) {
00163     for (int i = 0; i<13;i++) {
00164         if (str[i].empty()) {
00165             return false;
00166         }
00167     }
00168     return true;
00169 }

```

7.28 Army.cpp

```

00001 //
00002 // Created by bartl on 14.05.2026.
00003 //
00004
00005 #include <core/Army.hpp>
00006
00007 #include "warriors/Archer.hpp"
00008 #include "warriors/Cavalryman.hpp"
00009 #include "warriors/Knight.hpp"
00010 #include "utils/Random.hpp"
00011
00012
00013 Army::Army(const FractionConfig& config):
00014     name(config.fraction), attackBonus(0), defenseBonus(0), moraleLevel(50),
00015     knightStats(config.knight_stats),
00016     archerStats(config.archer_stats),
00017     cavalryStats(config.cavalry_stats)
00018 {
00019
00020
00021     if (!config.valid) {
00022         throw std::runtime_error("Invalid config");
00023     }
00024
00025     for (int i = 0; i < config.knight_count; i++) {
00026         warriors.emplace_back(std::make_shared<Knight>(config.knight_stats));
00027     }
00028
00029     for (int i = 0; i < config.archer_count; i++) {
00030         warriors.emplace_back(std::make_shared<Archer>(config.archer_stats));
00031     }
00032
00033     for (int i = 0; i < config.cavalry_count; i++) {
00034         warriors.emplace_back(std::make_shared<Cavalryman>(config.cavalry_stats));
00035     }
00036 }
00037
00038 int Army::count_AliveWarriors() const {
00039     int count = 0;
00040     for (const std::shared_ptr<Warrior>& w : warriors) {
00041         if (w->is_Alive()) {
00042             count++;
00043         }
00044     }
00045     return count;
00046 }
00047
00048 int Army::count_AliveKnights() const {
00049     int count{};
00050
00051     for (const auto& w : warriors) {
00052         if (w->is_Alive() && dynamic_cast<Knight*>(w.get())) {
00053             count++;
00054         }
00055     }
00056     return count;
00057 }
00058
00059 int Army::count_AliveArchers() const {
00060     int count{};
00061
00062     for (const auto& w : warriors) {
00063         if (w->is_Alive() && dynamic_cast<Archer*>(w.get())) {
00064             count++;
00065         }
00066     }
00067     return count;
00068 }
00069
00070 int Army::count_AliveCavalry() const {
00071     int count{};
00072
00073     for (const auto& w : warriors) {
00074         if (w->is_Alive() && dynamic_cast<Cavalryman*>(w.get())) {
00075             count++;
00076         }
00077     }
00078     return count;
00079 }
00080
00081 // Getters
00082
00083 size_t Army::get_ArmySize() const {
00084     return warriors.size();

```

```

00085 }
00086
00087 const std::vector<std::shared_ptr<Warrior>& Army::get_Warriors() const{
00088     return warriors;
00089 }
00090
00091 Warrior& Army::get_RandomAliveWarrior() {
00092     std::vector<Warrior*> aliveWarriors;
00093
00094     for (std::shared_ptr<Warrior>& w : warriors) {
00095         if (w->is_Alive()) {
00096             aliveWarriors.emplace_back(w.get());
00097         }
00098     }
00099
00100     if (aliveWarriors.empty()) {
00101         throw std::runtime_error("No alive warrior");
00102     }
00103
00104     int randomWarrior_id = Random::random_Int(0, static_cast<int>(aliveWarriors.size()-1);
00105     return *aliveWarriors[randomWarrior_id];
00106 }
00107
00108 // Setters
00109
00110 void Army::increase_Morale(int amount) {
00111     moraleLevel += amount;
00112     moraleLevel = std::min(moraleLevel,100);
00113 }
00114
00115 void Army::decrease_Morale(int amount) {
00116     moraleLevel -= amount;
00117     moraleLevel = std::max(moraleLevel,0);
00118 }
00119
00120
00121 void Army::add_Reinforcements(int amount) {
00122     for(int i = 0; i < amount; i++){
00123         int type = Random::random_Int(0,2);
00124
00125         switch(type){
00126             case 0:
00127                 warriors.emplace_back(std::make_shared<Knight>(knightStats));
00128                 break;
00129             case 1:
00130                 warriors.emplace_back(std::make_shared<Archer>(archerStats));
00131                 break;
00132             case 2:
00133                 warriors.emplace_back(std::make_shared<Cavalryman>(cavalryStats));
00134                 break;
00135         }
00136     }
00137 }
00138
00139 // Getter
00140
00141 int Army::get_moraleLevel() const {
00142     return moraleLevel;
00143 }
00144
00145 double Army::get_MoraleModifier() const {
00146     if (moraleLevel >= 70) {
00147         return 1.2;
00148     }
00149     if (moraleLevel <= 30) {
00150         return 0.8;
00151     }
00152
00153     return 1.0;
00154 }
00155
00156 const std::string& Army::get_Name() const {
00157     return name;
00158 }

```

7.29 Battle.cpp

```

00001 #include <random>
00002 #include <iostream>
00003
00004 #include <core/Battle.hpp>
00005 #include <utils/Random.hpp>
00006 #include <events/CommanderDeath.hpp>

```

```

00007 #include <events/AlliedHelp.hpp>
00008 #include <events/PanicEvent.hpp>
00009
00010
00011 // Main loop of battle ( attack and defend )
00012
00013 Battle::Battle(Army& HussarsArmy, Army& TeutonicArmy):
00014 {
00015     Hussars(HussarsArmy), Teutonic(TeutonicArmy), round(1), winner(), weather(), stats(), eventsTriggered(0), maxEvents(3) {
00016         randomEvents.push_back(std::make_unique<CommanderDeath>());
00017         randomEvents.push_back(std::make_unique<AlliedHelp>());
00018         randomEvents.push_back(std::make_unique<PanicEvent>());
00019     }
00020 }
00021
00022 // Start Battle loop
00023 void Battle::start_Battle() {
00024     if (is_finished()) {
00025         do_Round();
00026         finishBattle();
00027     }
00028 }
00029
00030 // Logic of doing round
00031 void Battle::do_Round() {
00032     Warrior& hussars(Hussars.get_RandomAliveWarrior());
00033     Warrior& teutonic(Teutonic.get_RandomAliveWarrior());
00034     WeatherType weatherModifier(weather.get_WeatherType());
00035     double damage_dealt{};
00036     if (Random::random_Int(0,1)==0) {
00037         damage_dealt = teutonic.get_Health();
00038         currentEvent = hussars.get_Type() + " attacked " + teutonic.get_Type();
00039         hussars.attack_Energy(teutonic, Hussars.get_MoraleModifier(), weatherModifier);
00040         damage_dealt -= teutonic.get_Health();
00041         stats.add_HussarsDamage(damage_dealt);
00042     }
00043     if (teutonic.is_Alive()) {
00044         damage_dealt = hussars.get_Health();
00045         currentEvent = teutonic.get_Type() + " counter-attacked " + hussars.get_Type();
00046         teutonic.attack_Energy(hussars, Teutonic.get_MoraleModifier(), weatherModifier);
00047         damage_dealt -= hussars.get_Health();
00048         stats.add_TeutonicDamage(damage_dealt);
00049     }
00050 }
00051 else {
00052     damage_dealt = hussars.get_Health();
00053     currentEvent = teutonic.get_Type() + " attacked " + hussars.get_Type();
00054     teutonic.attack_Energy(hussars, Teutonic.get_MoraleModifier(), weatherModifier);
00055     damage_dealt -= hussars.get_Health();
00056     stats.add_TeutonicDamage(damage_dealt);
00057 }
00058 if (hussars.is_Alive()) {
00059     damage_dealt = teutonic.get_Health();
00060     currentEvent = hussars.get_Type() + " counter-attacked " + teutonic.get_Type();
00061     hussars.attack_Energy(teutonic, Hussars.get_MoraleModifier(), weatherModifier);
00062     damage_dealt -= teutonic.get_Health();
00063     stats.add_HussarsDamage(damage_dealt);
00064 }
00065 if (!hussars.is_Alive()) {
00066     Hussars.decrease_Morale(5);
00067     Teutonic.increase_Morale(5);
00068     stats.add_TeutonicKill();
00069     currentEvent = teutonic.get_Type() + " killed " + hussars.get_Type();
00070 }
00071 if (!teutonic.is_Alive()) {
00072     Teutonic.decrease_Morale(5);
00073     Hussars.increase_Morale(5);
00074 }

```

```

00093
00094     stats.add_HussarsKill();
00095
00096     currentEvent = hussars.get_Type() + " killed " + teutonic.get_Type();
00097
00098 }
00099
00100 round++;
00101
00102 if (round > 20 && eventsTriggered<maxEvents) {
00103     if (Random::random_Int(1,100)<=3) {
00104         triggerRandomEvent();
00105         eventsTriggered++;
00106     }
00107 }
00108
00109 notify();
00110 }
00111
00112 bool Battle::is_finished() const {
00113     return Hussars.count_AliveWarriors()==0 || Teutonic.count_AliveWarriors()==0;
00114 }
00115
00116 void Battle::finishBattle() {
00117     if (Hussars.count_AliveWarriors(>0) {
00118         winner = "Hussars";
00119     }
00120     else {
00121         winner = "Teutonic";
00122     }
00123 }
00124
00125 void Battle::notify() {
00126     for (BattleObserver* observer : observers) {
00127         observer->update(*this);
00128     }
00129 }
00130
00131 // Getter
00132 int Battle::get_Rounds() const {
00133     return round;
00134 }
00135
00136 std::string Battle::get_Winner() const {
00137     return winner;
00138 }
00139
00140 const Army& Battle::get_HussarsArmy() const {
00141     return Hussars;
00142 }
00143
00144 const Army& Battle::get_TeutonicArmy() const {
00145     return Teutonic;
00146 }
00147
00148 std::string Battle::get_Weather() const {
00149     return weather.to_String();
00150 }
00151
00152 const BattleStats& Battle::get_Stats() const {
00153     return stats;
00154 }
00155
00156 void Battle::attach(BattleObserver* observer) {
00157     observers.push_back(observer);
00158 }
00159
00160 const std::string& Battle::get_CurrentEvent() const {
00161     return currentEvent;
00162 }
00163
00164 void Battle::triggerRandomEvent() {
00165     int id = Random::random_Int(0, static_cast<int>(randomEvents.size()) - 1);
00166
00167     Army& targetArmy = Random::random_Int(0,1) ? Hussars : Teutonic;
00168
00169     randomEvents[id]->apply(targetArmy);
00170
00171     currentEvent = targetArmy.get_Name() + ": " + randomEvents[id]->getName();
00172
00173     notify();
00174 }

```


7.30 BattleStats.cpp

```

00001 //
00002 // Created by bart on 30.05.2026.
00003 //
00004
00005
00006 #include "core/BattleStats.hpp"
00007
00008
00009 void BattleStats::add_HussarsDamage(double damage) {
00010     HussarsDamage_Dealt+=damage;
00011 }
00012
00013 void BattleStats::add_TeutonicDamage(double damage) {
00014     TeutonicDamage_Dealt+=damage;
00015 }
00016
00017
00018 void BattleStats::add_HussarsKill() {
00019     HussarsKills+=1;
00020 }
00021
00022 void BattleStats::add_TeutonicKill() {
00023     TeutonicKills+=1;
00024 }
00025
00026
00027 // Getters
00028
00029 double BattleStats::get_HussarsDamage_Dealt() const {
00030     return HussarsDamage_Dealt;
00031 }
00032
00033 double BattleStats::get_TeutonicDamage_Dealt() const {
00034     return TeutonicDamage_Dealt;
00035 }
00036
00037
00038 int BattleStats::get_HussarsKills() const {
00039     return HussarsKills;
00040 }
00041
00042 int BattleStats::get_TeutonicKills() const {
00043     return TeutonicKills;
00044 }
00045
00046

```

7.31 AlliedHelp.cpp

```

00001 //
00002 // Created by bart on 1.06.2026.
00003 //
00004
00005 #include "events/AlliedHelp.hpp"
00006 #include "utils/Random.hpp"
00007
00008 void AlliedHelp::apply(Army& army) {
00009     army.add_Reinforcements(Random::random_Int(10,500));
00010     army.increase_Morale(15);
00011 }
00012
00013 std::string AlliedHelp::getName() const {
00014     return "Allied Reinforcement";
00015 }

```

7.32 CommanderDeath.cpp

```

00001 //
00002 // Created by bart on 1.06.2026.
00003 //
00004
00005 #include <events/CommanderDeath.hpp>
00006
00007 void CommanderDeath::apply(Army& army) {
00008     army.decrease_Morale(20);

```

```

00009 }
00010
00011 std::string CommanderDeath::getName() const {
00012     return "Commander Death";
00013 }

```

7.33 PanicEvent.cpp

```

00001 //
00002 // Created by bart on 1.06.2026.
00003 //
00004
00005 #include <events/PanicEvent.hpp>
00006
00007 void PanicEvent::apply(Army& army) {
00008     army.decrease_Morale(10);
00009 }
00010
00011 std::string PanicEvent::getName() const {
00012     return "Army Panic";
00013 }

```

7.34 Weather.cpp

```

00001 //
00002 // Created by bart1 on 15.05.2026.
00003 //
00004
00005 #include <events/Weather.hpp>
00006 #include <utils/Random.hpp>
00007
00008
00009 Weather::Weather() {
00010
00011     switch (Random::random_Int(0, 3)) {
00012     case 0:
00013         weatherType = WeatherType::SUNNY;
00014         break;
00015     case 1:
00016         weatherType = WeatherType::RAINY;
00017         break;
00018     case 2:
00019         weatherType = WeatherType::FOGGY;
00020         break;
00021     case 3:
00022         weatherType = WeatherType::SNOWY;
00023         break;
00024     default:
00025         weatherType = WeatherType::SUNNY;
00026         break;
00027     }
00028 }
00029
00030 // Get weather type for modify stats of each subclass warrior
00031 WeatherType Weather::get_WeatherType() const {
00032     return weatherType;
00033 }
00034
00035
00036 // Switch to string for showing in program and saving result
00037 std::string Weather::to_String() const {
00038     switch (weatherType) {
00039     case WeatherType::SUNNY:
00040         return "SUNNY";
00041     case WeatherType::RAINY:
00042         return "RAINY";
00043     case WeatherType::FOGGY:
00044         return "FOGGY";
00045     case WeatherType::SNOWY:
00046         return "SNOWY";
00047     default:
00048         return "UNKNOWN";
00049     }
00050 }

```

7.35 BattlePanel.cpp

```

00001 //
00002 // Created by bart on 31.05.2026.
00003 //
00004
00005 #include "gui/BattlePanel.hpp"
00006
00007 #include <QVBoxLayout>
00008 #include <QGroupBox>
00009 #include <QPixmap>
00010 #include <QHBoxLayout>
00011
00012
00013
00014 // Update for each round stats
00015 void BattlePanel::update(const Battle &battle) {
00016     setRounds(battle.get_Rounds());
00017
00018     setWeather(QString::fromStdString(battle.get_Weather()));
00019
00020     setHussarsMorale(battle.get_HussarsArmy().get_moraleLevel());
00021     setTeutonicMorale(battle.get_TeutonicArmy().get_moraleLevel());
00022
00023     setCurrentEvent(QString::fromStdString(battle.get_CurrentEvent()));
00024
00025 }
00026
00027 // Gui look of battle panel
00028 BattlePanel::BattlePanel(QWidget* parent) : QWidget(parent), battle(nullptr) {
00029
00030     QVBoxLayout* layout = new QVBoxLayout;
00031     QGroupBox* battleGroup = new QGroupBox("Battle Information", this);
00032     QVBoxLayout* battleLayout = new QVBoxLayout;
00033
00034     battleNumberLabel = new QLabel(this);
00035     battleNumberLabel->setText("Battle Number: -/-");
00036     battleLayout->addWidget(battleNumberLabel);
00037
00038     roundLabel = new QLabel(this);
00039     roundLabel->setText("Rounds: -");
00040     battleLayout->addWidget(roundLabel);
00041
00042     weatherLabel = new QLabel(this);
00043     weatherLabel->setText("Weather: -");
00044     battleLayout->addWidget(weatherLabel);
00045
00046     HussarsMoraleLabel = new QLabel(this);
00047     HussarsMoraleLabel->setText("Hussars Morale: -");
00048     battleLayout->addWidget(HussarsMoraleLabel);
00049
00050     TeutonicMoraleLabel = new QLabel(this);
00051     TeutonicMoraleLabel->setText("Teutonic Morale: -");
00052     battleLayout->addWidget(TeutonicMoraleLabel);
00053
00054     VictoryChanceLabel = new QLabel(this);
00055     VictoryChanceLabel->setText("Victory Chance:");
00056     battleLayout->addWidget(VictoryChanceLabel);
00057
00058     HussarsProbabilityLabel = new QLabel(this);
00059     HussarsProbabilityLabel->setText("Hussars: -%");
00060     battleLayout->addWidget(HussarsProbabilityLabel);
00061
00062     TeutonicProbabilityLabel = new QLabel(this);
00063     TeutonicProbabilityLabel->setText("Teutonic: -%");
00064     battleLayout->addWidget(TeutonicProbabilityLabel);
00065
00066     currentEventLabel = new QLabel(this);
00067     currentEventLabel->setText("Current Event: -");
00068     battleLayout->addWidget(currentEventLabel);
00069
00070
00071     QFont winnerFont;
00072     winnerFont.setBold(true);
00073     winnerFont.setPointSize(11);
00074
00075     winnerLabel = new QLabel(this);
00076     winnerLabel->setText("Current Winner: -");
00077     winnerLabel->setFont(winnerFont);
00078     battleLayout->addWidget(winnerLabel);
00079
00080     battleLayout->addStretch();
00081     battleGroup->setLayout(battleLayout);
00082     layout->addWidget(battleGroup);
00083     setLayout(layout);
00084 }

```

```

00085
00086
00087 // Setters
00088
00089 void BattlePanel::setBattleNumber(int current, int total) {
00090     battleNumberLabel->setText(
00091         QString("Battle %1 / %2")
00092         .arg(current)
00093         .arg(total)
00094     );
00095 }
00096
00097 void BattlePanel::setRounds(int rounds) {
00098     roundLabel->setText("Rounds: " + QString::number(rounds));
00099 }
00100 void BattlePanel::setWeather(const QString& weather) {
00101     weatherLabel->setText("Weather: " + weather);
00102 }
00103
00104 void BattlePanel::setHussarsMorale(int morale) {
00105     HussarsMoraleLabel->setText("Hussars Morale: " + QString::number(morale));
00106     if (morale >= 70)
00107         HussarsMoraleLabel->setStyleSheet("color: green;");
00108     else if (morale <= 30)
00109         HussarsMoraleLabel->setStyleSheet("color: red;");
00110     else
00111         HussarsMoraleLabel->setStyleSheet("color: orange;");
00112 }
00113
00114 void BattlePanel::setTeutonicMorale(int morale) {
00115     TeutonicMoraleLabel->setText("Teutonic Morale: " + QString::number(morale));
00116     if (morale >= 70)
00117         TeutonicMoraleLabel->setStyleSheet("color: green;");
00118     else if (morale <= 30)
00119         TeutonicMoraleLabel->setStyleSheet("color: red;");
00120     else
00121         TeutonicMoraleLabel->setStyleSheet("color: orange;");
00122 }
00123
00124 void BattlePanel::setHussarsProbability(double probability) {
00125     HussarsProbabilityLabel->setText(QString("Hussars: %1%")
00126         .arg(QString::number(probability, 'f', 1)));
00127 }
00128
00129 void BattlePanel::setTeutonicProbability(double probability) {
00130     TeutonicProbabilityLabel->setText(QString("Teutonic: %1%")
00131         .arg(QString::number(probability, 'f', 1)));
00132 }
00133
00134 void BattlePanel::setCurrentEvent(const QString &event) {
00135     currentEventLabel->setText("Current event: " + event);
00136 }
00137
00138 void BattlePanel::setWinner(const QString& winner) {
00139     winnerLabel->setText("Current Winner: " + winner);
00140
00141     if (winner == "HUSSARS") {
00142         winnerLabel->setStyleSheet("color: green");
00143     }
00144     else if (winner == "TEUTONIC") {
00145         winnerLabel->setStyleSheet("color: red");
00146     }
00147     else {
00148         winnerLabel->setStyleSheet("color: yellow");
00149     }
00150 }

```

7.36 ControlPanel.cpp

```

00001 //
00002 // Created by bart on 31.05.2026.
00003 //
00004
00005 #include "gui/ControlPanel.hpp"
00006
00007 #include <QBoxLayout>
00008
00009 ControlPanel::ControlPanel(QWidget* parent): QWidget(parent) {
00010
00011     QVBoxLayout* layout = new QVBoxLayout;
00012
00013     startButton=new QPushButton("Start", this);
00014     startButton->setMinimumHeight(50);

```

```

00015     layout->addWidget(startButton);
00016
00017     loadConfigButton=new QPushButton("Load", this);
00018     loadConfigButton->setMinimumHeight(50);
00019     layout->addWidget(loadConfigButton);
00020
00021     exitButton=new QPushButton("Exit", this);
00022     exitButton->setMinimumHeight(50);
00023     layout->addWidget(exitButton);
00024
00025     speedComboBox=new QComboBox(this);
00026     speedComboBox->addItem("Slow");
00027     speedComboBox->addItem("Normal");
00028     speedComboBox->addItem("Fast");
00029
00030     speedComboBox->setCurrentText("Normal");
00031
00032     layout->addWidget(speedComboBox);
00033
00034     this->setLayout(layout);
00035 }
00036
00037
00038 QPushButton* ControlPanel::get_StartButton() const {
00039     return startButton;
00040 }
00041
00042 QPushButton* ControlPanel::get_LoadButton() const {
00043     return loadConfigButton;
00044 }
00045
00046 QPushButton* ControlPanel::get_ExitButton() const {
00047     return exitButton;
00048 }
00049
00050 QComboBox* ControlPanel::get_SpeedComboBox() const {
00051     return speedComboBox;
00052 }

```

7.37 HistoryPanel.cpp

```

00001 //
00002 // Created by bart on 31.05.2026.
00003 //
00004
00005 #include "gui/HistoryPanel.hpp"
00006
00007 #include <QBoxLayout>
00008 #include <QWidget>
00009
00010 HistoryPanel::HistoryPanel(QWidget *parent) {
00011
00012     QVBoxLayout* layout = new QVBoxLayout();
00013     QVBoxLayout* historyLayout = new QVBoxLayout();
00014
00015     QGroupBox* historyGroup = new QGroupBox("Battle History", this);
00016
00017     historyList = new QListWidget(this);
00018
00019     historyLayout->addWidget(historyList);
00020     historyGroup->setLayout(historyLayout);
00021     layout->addWidget(historyGroup);
00022     setLayout(layout);
00023
00024 }
00025
00026 void HistoryPanel::clearHistoryList() {
00027     historyList->clear();
00028 }
00029
00030 // Setter
00031 void HistoryPanel::addBattleResult(const QString& result) {
00032     historyList->addItem(result);
00033 }

```

7.38 HussarsPanel.cpp

```

00001 //

```

```

00002 // Created by bart on 1.06.2026.
00003 //
00004
00005 #include "gui/HussarsPanel.hpp"
00006
00007 #include <complex>
00008 #include <QGroupBox>
00009 #include <QHBoxLayout>
00010
00011
00012 HussarsPanel::HussarsPanel(QWidget* parent) {
00013     QVBoxLayout* layout = new QVBoxLayout;
00014     QGroupBox* hussarsPanel = new QGroupBox("Hussars Stats");
00015     QVBoxLayout* groupLayout = new QVBoxLayout;
00016
00017     QFont armyFont;
00018     armyFont.setPointSize(14);
00019     armyFont.setBold(true);
00020
00021     hussarsLabel = new QLabel("HUSSARS", this);
00022     hussarsLabel->setFont(armyFont);
00023     hussarsLabel->setAlignment(Qt::AlignCenter);
00024     groupLayout->addWidget(hussarsLabel);
00025
00026
00027     hussarsIMG = new QLabel(this);
00028     hussarsIMG->setAlignment(Qt::AlignCenter);
00029     groupLayout->addWidget(hussarsIMG);
00030     QPixmap hussarsPixmap("../assets/images/hussarimg.png");
00031     hussarsIMG->setPixmap(hussarsPixmap.scaled(140,140, Qt::KeepAspectRatio,
00032 Qt::SmoothTransformation));
00033
00034
00035     aliveHussarsLabel = new QLabel("Alive: -");
00036     groupLayout->addWidget(aliveHussarsLabel);
00037
00038     aliveKnightsLabel = new QLabel("Knights: -");
00039     groupLayout->addWidget(aliveKnightsLabel);
00040     aliveArchersLabel = new QLabel("Archers: -");
00041     groupLayout->addWidget(aliveArchersLabel);
00042     aliveCavalrymansLabel = new QLabel("Cavalrymans: -");
00043     groupLayout->addWidget(aliveCavalrymansLabel);
00044
00045
00046     hussarsDamageLabel = new QLabel("Damage: -");
00047     groupLayout->addWidget(hussarsDamageLabel);
00048     hussarsKillsLabel = new QLabel("Kills: -");
00049     groupLayout->addWidget(hussarsKillsLabel);
00050
00051     hussarsPanel->setLayout(groupLayout);
00052     layout->addWidget(hussarsPanel);
00053
00054     setLayout(layout);
00055
00056
00057 }
00058
00059 // Setters
00060
00061 void HussarsPanel::setAlive(int alive) {
00062     aliveHussarsLabel->setText(
00063         "Alive: " + QString::number(alive)
00064     );
00065 }
00066
00067 void HussarsPanel::setKnights(int knights) {
00068     aliveKnightsLabel->setText("Knights: " + QString::number(knights));
00069 }
00070
00071 void HussarsPanel::setArchers(int archers) {
00072     aliveArchersLabel->setText("Archers: " + QString::number(archers));
00073 }
00074
00075 void HussarsPanel::setCavalrymans(int cavalrymans) {
00076     aliveCavalrymansLabel->setText("Cavalrymans: " + QString::number(cavalrymans));
00077 }
00078
00079 void HussarsPanel::setDamage(double damage) {
00080     hussarsDamageLabel->setText(
00081         "Damage: " + QString::number(damage)
00082     );
00083 }
00084
00085 void HussarsPanel::setKills(int kills) {
00086     hussarsKillsLabel->setText(
00087         "Kills: " + QString::number(kills)

```

```
00088     );
00089 }
```

7.39 MainWindow.cpp

```
00001 //
00002 // Created by bart on 31.05.2026.
00003 //
00004
00005 #include <iostream>
00006 #include <string>
00007 #include <fstream>
00008
00009 #include "gui/MainWindow.hpp"
00010
00011 #include "gui/BattlePanel.hpp"
00012 #include "gui/ControlPanel.hpp"
00013 #include "gui/HistoryPanel.hpp"
00014 #include <config/ConfigLoader.hpp>
00015 #include <core/Battle.hpp>
00016 #include <reports/BattleReport.hpp>
00017
00018 #include <QPushButton>
00019 #include <QDebug>
00020 #include <QFileDialog>
00021 #include <QMessageBox>
00022
00023
00024
00025 // Gui
00026
00027 MainWindow::MainWindow(QWidget *parent): QMainWindow(parent) {
00028     setWindowTitle(" ");
00029     resize(1000, 700);
00030
00031     // Change design of window
00032     setStyleSheet (
00033         "QMainWindow {"
00034             "background-image: url(../assets/images/backgroundbattle.jpg);"
00035             "background-position: center;"
00036             "background-repeat: no-repeat;"
00037         "}"
00038
00039         "QGroupBox {"
00040             "background-color: rgba(30,30,30,180);"
00041             "color: white;"
00042             "border: 2px solid #444;"
00043             "border-radius: 5px;"
00044             "margin-top: 10px;"
00045             "font-weight: bold;"
00046         "}"
00047
00048         "QGroupBox::title {"
00049             "background-color: rgba(30,30,30,180);"
00050             "color: white;"
00051             "subcontrol-origin: margin;"
00052             "left: 10px;"
00053             "padding: 0 5px;"
00054         "}"
00055
00056         "QListWidget {"
00057             "background-color: rgba(20,20,20,180);"
00058             "color: white;"
00059             "border: 1px solid #555;"
00060         "}"
00061
00062         "QLabel {"
00063             "color: white;"
00064         "}"
00065
00066         "QPushButton {"
00067             "background-color: #303030;"
00068             "color: white;"
00069             "padding: 8px;"
00070             "border: 1px solid #555;"
00071         "}"
00072
00073         "QMessageBox {"
00074             "background-color: #1e1e1e;"
00075         "}"
00076
00077         "QMessageBox QLabel {"
00078             "color: white;"
```

```

00079     " margin: 0px;"
00080     " padding: 0px;"
00081     "}"
00082
00083     "QMessageBox QPushButton {"
00084     " background-color: #303030;"
00085     " color: white;"
00086     " border: 1px solid #555;"
00087     " padding: 6px;"
00088     " min-width: 80px;"
00089     "}"
00090     );
00091
00092     QWidget* centralWidget = new QWidget(this);
00093
00094     QLabel* title = new QLabel("Simulation Battle of Grunwald", this);
00095     title->setAlignment(Qt::AlignCenter);
00096     QFont font;
00097     font.setPointSize(20);
00098     font.setBold(true);
00099     title->setFont(font);
00100
00101     QVBoxLayout* mainLayout = new QVBoxLayout;
00102     QHBoxLayout* topLayout = new QHBoxLayout;
00103     QHBoxLayout* bottomLayout = new QHBoxLayout;
00104
00105
00106     hussarsPanel = new HussarsPanel(centralWidget);
00107     teutonicPanel = new TeutonicPanel(centralWidget);
00108
00109     battlePanel = new BattlePanel(centralWidget);
00110     controlPanel = new ControlPanel(centralWidget);
00111     historyPanel = new HistoryPanel(centralWidget);
00112
00113     battleTimer = new QTimer(this);
00114     battleTimer->setInterval(200);
00115
00116     mainLayout->addWidget(title);
00117
00118     /*
00119     topLayout->addWidget(statisticsPanel);
00120     topLayout->addWidget(battlePanel);
00121     topLayout->addWidget(controlPanel);
00122     */
00123
00124     topLayout->addWidget(hussarsPanel);
00125     topLayout->addWidget(battlePanel);
00126     topLayout->addWidget(teutonicPanel);
00127
00128     bottomLayout->addWidget(historyPanel);
00129     bottomLayout->addWidget(controlPanel);
00130
00131
00132     mainLayout->addLayout(topLayout);
00133     mainLayout->addLayout(bottomLayout);
00134
00135     mainLayout->setStretch(0, 0); // title
00136     mainLayout->setStretch(1, 3); // mid
00137     mainLayout->setStretch(2, 1); // history
00138
00139     centralWidget->setLayout(mainLayout);
00140     setCentralWidget(centralWidget);
00141
00142     // Connect buttons
00143     connect (
00144         controlPanel->get_StartButton(),
00145         &QPushButton::clicked,
00146         this,
00147         &MainWindow::startBattle
00148     );
00149
00150     connect (
00151         controlPanel->get_LoadButton(),
00152         &QPushButton::clicked,
00153         this,
00154         &MainWindow::loadConfig
00155     );
00156
00157     connect (
00158         controlPanel->get_ExitButton(),
00159         &QPushButton::clicked,
00160         this,
00161         &QWidget::close
00162     );
00163
00164     connect (
00165         battleTimer,

```



```

00166         &QTimer::timeout,
00167         this,
00168         &MainWindow::battleStep
00169     );
00170
00171     connect(
00172         controlPanel->get_SpeedComboBox(),
00173         &QComboBox::currentTextChanged,
00174         this,
00175         &MainWindow::changeSpeed
00176     );
00177
00178 }
00179
00180 void MainWindow::startBattle()
00181 {
00182     historyPanel->clearHistoryList();
00183
00184     if(configPath.isEmpty())
00185     {
00186         QMessageBox::warning(
00187             this,
00188             "No config",
00189             "Load configuration first."
00190         );
00191         return;
00192     }
00193
00194     loadedConfig = ConfigLoader::LoadFromFile(configPath.toStdString(), false);
00195
00196     QString defaultReportName = QFileInfo(configPath).baseName()+"_report.csv";
00197
00198     QString savePath = QFileDialog::getSaveFileName(this,
00199                                                     "Save Battle Report",
00200                                                     "../assets/reports/" + defaultReportName,
00201                                                     "CSV Files (*.csv)");
00202
00203     if (savePath.isEmpty()) {
00204         return;
00205     }
00206
00207     ReportFilename = savePath.toStdString();
00208
00209     std::ofstream file(ReportFilename);
00210     file<<"Battle, Winner, Rounds, Hussars Alive, Teutonic Alive, Hussars Total Damage, Teutonic Total
Damage, Hussars Kills, Teutonic Kills, Weather, Status, Date\n";
00211     file.close();
00212
00213
00214     currentBattleIndex = 0;
00215
00216     hussarsWins = 0;
00217     teutonicWins = 0;
00218
00219     startNextBattle();
00220 }
00221
00222 void MainWindow::loadConfig() {
00223     QString filePath = QFileDialog::getOpenFileName(this,
00224             "Select config to open",
00225             "../assets/configs",
00226             "CSV Files (*.csv)"
00227     );
00228
00229     if (!filePath.isEmpty()) {
00230         configPath = filePath;
00231         qDebug() << "Loaded config: " << configPath;
00232         QMessageBox::information(
00233             this,
00234             "Config loaded",
00235             "Configuration loaded successfully."
00236         );
00237     }
00238 }
00239
00240 void MainWindow::battleStep() {
00241     currentBattle->do_Round();
00242
00243     // Update count alive warriors
00244     hussarsPanel->setAlive(currentBattle->get_HussarsArmy().count_AliveWarriors());
00245     teutonicPanel->setAlive(currentBattle->get_TeutonicArmy().count_AliveWarriors());
00246
00247     // Update for each subclass count of alive units
00248     hussarsPanel->setKnights(currentBattle->get_HussarsArmy().count_AliveKnights());
00249     hussarsPanel->setArchers(currentBattle->get_HussarsArmy().count_AliveArchers());
00250     hussarsPanel->setCavalrymans(currentBattle->get_HussarsArmy().count_AliveCavalry());
00251

```

```

00252     teutonicPanel->setKnights(currentBattle->get_TeutonicArmy().count_AliveKnights());
00253     teutonicPanel->setArchers(currentBattle->get_TeutonicArmy().count_AliveArchers());
00254     teutonicPanel->setCavalrymans(currentBattle->get_TeutonicArmy().count_AliveCavalry());
00255
00256     // Update total damage and kills
00257     hussarsPanel->setDamage(currentBattle->get_Stats().get_HussarsDamage_Dealt());
00258     teutonicPanel->setDamage(currentBattle->get_Stats().get_TeutonicDamage_Dealt());
00259     hussarsPanel->setKills(currentBattle->get_Stats().get_HussarsKills());
00260     teutonicPanel->setKills(currentBattle->get_Stats().get_TeutonicKills());
00261
00262
00263     int aliveHussars = currentBattle->get_HussarsArmy().count_AliveWarriors();
00264     int aliveTeutonic = currentBattle->get_TeutonicArmy().count_AliveWarriors();
00265
00266     int moraleHussars = currentBattle->get_HussarsArmy().get_moraleLevel();
00267     int moraleTeutonic = currentBattle->get_TeutonicArmy().get_moraleLevel();
00268
00269     int killsHussars = currentBattle->get_Stats().get_HussarsKills();
00270     int killsTeutonic = currentBattle->get_Stats().get_TeutonicKills();
00271
00272     // Probability update and logic
00273     double hussarsPower = aliveHussars*2+moraleHussars+killsHussars;
00274     double teutonicPower = aliveTeutonic*2+moraleTeutonic+killsTeutonic;
00275
00276     double totalPower = hussarsPower+teutonicPower;
00277
00278     if (totalPower > 0) {
00279         double hussarsProbability = (hussarsPower/totalPower)*100.0;
00280         double teutonicProbability = (teutonicPower/totalPower)*100.0;
00281
00282         battlePanel->setHussarsProbability(hussarsProbability);
00283         battlePanel->setTeutonicProbability(teutonicProbability);
00284
00285         double diff = std::abs(hussarsProbability-teutonicProbability);
00286
00287         if (diff < 1.0) {
00288             battlePanel->setWinner("DRAW");
00289         }
00290         else if (hussarsProbability > teutonicProbability) {
00291             battlePanel->setWinner("HUSSARS");
00292         }
00293         else{
00294             battlePanel->setWinner("TEUTONIC");
00295         }
00296     }
00297
00298
00299     if(currentBattle->is_finished())
00300     {
00301         battleTimer->stop();
00302
00303         currentBattle->finishBattle();
00304
00305         historyPanel->addBattleResult (
00306             QString("%1 | Rounds: %2 | Weather: %3 | Alive: %4 vs %5")
00307             .arg(QString::fromStdString(currentBattle->get_Winner()))
00308             .arg(currentBattle->get_Rounds())
00309             .arg(QString::fromStdString(currentBattle->get_Weather()))
00310             .arg(currentBattle->get_HussarsArmy().count_AliveWarriors())
00311             .arg(currentBattle->get_TeutonicArmy().count_AliveWarriors())
00312             );
00313
00314         BattleReport::save(*currentBattle, ReportFilename, (currentBattleIndex/2)+1);
00315
00316         if(currentBattle->get_Winner() == "Hussars")
00317         {
00318             hussarsWins++;
00319         }
00320         else
00321         {
00322             teutonicWins++;
00323         }
00324
00325         currentBattleIndex += 2;
00326
00327         delete currentBattle;
00328         currentBattle = nullptr;
00329
00330         startNextBattle();
00331
00332     }
00333 }
00334
00335 void MainWindow::startNextBattle()
00336 {
00337     if(currentBattleIndex >= loadedConfig.size() - 1)
00338     {

```

```

00339     historyPanel->addBattleResult (
00340         "=== Campaign Finished ==="
00341     );
00342
00343     historyPanel->addBattleResult (
00344         QString("Hussars Wins: %1")
00345         .arg(hussarsWins)
00346     );
00347
00348     historyPanel->addBattleResult (
00349         QString("Teutonic Wins: %1")
00350         .arg(teutonicWins)
00351     );
00352
00353     historyPanel->addBattleResult (
00354         QString("Total Battles: %1")
00355         .arg(hussarsWins + teutonicWins)
00356     );
00357
00358     QString champion =
00359         (hussarsWins > teutonicWins)
00360         ? "HUSSARS"
00361         : "TEUTONIC";
00362
00363     historyPanel->addBattleResult (
00364         QString("Campaign Winner: %1")
00365         .arg(champion)
00366     );
00367
00368     return;
00369 }
00370
00371 try {
00372
00373     battlePanel->setBattleNumber((currentBattleIndex/2)+1, loadedConfig.size()/2);
00374
00375     Army Hussars(loadedConfig[currentBattleIndex]);
00376
00377     Army Teutonic(loadedConfig[currentBattleIndex + 1]);
00378
00379     currentBattle = new Battle(Hussars, Teutonic);
00380
00381     currentBattle->attach(battlePanel);
00382
00383     battleTimer->start();
00384 }
00385 catch (const std::exception& e) {
00386     BattleReport::save_skipped(ReportFilename, (currentBattleIndex/2)+1, e.what());
00387
00388     historyPanel->addBattleResult (
00389         QString("Battle %1 skipped: %2")
00390         .arg(currentBattleIndex)
00391         .arg(e.what())
00392     );
00393
00394     currentBattleIndex += 2;
00395
00396     startNextBattle();
00397 }
00398
00399 }
00400
00401 void MainWindow::changeSpeed() {
00402     QString speed = controlPanel->get_SpeedComboBox()->currentText();
00403
00404     if(speed == "Slow")
00405     {
00406         battleTimer->setInterval(500);
00407     }
00408     else if(speed == "Normal")
00409     {
00410         battleTimer->setInterval(200);
00411     }
00412     else if(speed == "Fast")
00413     {
00414         battleTimer->setInterval(50);
00415     }
00416 }

```

7.40 TeutonicPanel.cpp

```

00001 //
00002 // Created by bart on 1.06.2026.

```

```

00003 //
00004
00005 #include "gui/TeutonicPanel.hpp"
00006
00007 #include <QGroupBox>
00008 #include <QHBoxLayout>
00009
00010
00011 TeutonicPanel::TeutonicPanel(QWidget* parent) {
00012     QVBoxLayout* layout = new QVBoxLayout;
00013     QGroupBox* teutonicPanel = new QGroupBox("Teutonic Stats");
00014     QVBoxLayout* groupLayout = new QVBoxLayout;
00015
00016
00017     QFont armyFont;
00018     armyFont.setPointSize(14);
00019     armyFont.setBold(true);
00020
00021     teutonicLabel = new QLabel("TEUTONIC", this);
00022     teutonicLabel->setFont(armyFont);
00023     teutonicLabel->setAlignment(Qt::AlignCenter);
00024     groupLayout->addWidget(teutonicLabel);
00025
00026     teutonicIMG = new QLabel(this);
00027     teutonicIMG->setAlignment(Qt::AlignCenter);
00028     groupLayout->addWidget(teutonicIMG);
00029     QPixmap teutonicPixmap("../assets/images/teutonicimg.png");
00030     teutonicIMG->setPixmap(teutonicPixmap.scaled(140,140, Qt::KeepAspectRatio,
Qt::SmoothTransformation));
00031
00032
00033     aliveTeutonicLabel = new QLabel("Alive: -");
00034     groupLayout->addWidget(aliveTeutonicLabel);
00035
00036     aliveKnightsLabel = new QLabel("Knights: -");
00037     groupLayout->addWidget(aliveKnightsLabel);
00038     aliveArchersLabel = new QLabel("Archers: -");
00039     groupLayout->addWidget(aliveArchersLabel);
00040     aliveCavalrymansLabel = new QLabel("Cavalrymans: -");
00041     groupLayout->addWidget(aliveCavalrymansLabel);
00042
00043
00044     teutonicDamageLabel = new QLabel("Damage: -");
00045     groupLayout->addWidget(teutonicDamageLabel);
00046     teutonicKillsLabel = new QLabel("Kills: -");
00047     groupLayout->addWidget(teutonicKillsLabel);
00048
00049     teutonicPanel->setLayout(groupLayout);
00050     layout->addWidget(teutonicPanel);
00051
00052     setLayout(layout);
00053
00054 }
00055
00056 // Setters
00057
00058 void TeutonicPanel::setAlive(int alive) {
00059     aliveTeutonicLabel->setText(
00060         "Alive: " + QString::number(alive)
00061     );
00062 }
00063
00064 void TeutonicPanel::setKnights(int knights) {
00065     aliveKnightsLabel->setText("Knights: " + QString::number(knights));
00066 }
00067
00068 void TeutonicPanel::setArchers(int archers) {
00069     aliveArchersLabel->setText("Archers: " + QString::number(archers));
00070 }
00071
00072 void TeutonicPanel::setCavalrymans(int cavalrymans) {
00073     aliveCavalrymansLabel->setText("Cavalrymans: " + QString::number(cavalrymans));
00074 }
00075
00076 void TeutonicPanel::setDamage(double damage) {
00077     teutonicDamageLabel->setText(
00078         "Damage: " + QString::number(damage)
00079     );
00080 }
00081
00082 void TeutonicPanel::setKills(int kills) {
00083     teutonicKillsLabel->setText(
00084         "Kills: " + QString::number(kills)
00085     );
00086 }
00087

```

7.41 BattleReport.cpp

```

00001 //
00002 // Created by bart on 25.05.2026.
00003 //
00004
00005 #include <fstream>
00006 #include <chrono>
00007 #include <ctime>
00008
00009 #include "reports/BattleReport.hpp"
00010
00011 void BattleReport::save(const Battle& battle, const std::string& filename, int battleNumber) {
00012     auto now = std::chrono::system_clock::now();
00013     std::time_t currentTime = std::chrono::system_clock::to_time_t(now);
00014     std::string date = std::ctime(&currentTime);
00015     date.pop_back();
00016
00017     std::ofstream file;
00018     file.open(filename, std::ios::app);
00019     file<<battleNumber<<", "
00020     <<battle.get_Winner() <<", "
00021     <<battle.get_Rounds() <<", "
00022     <<battle.get_HussarsArmy().count_AliveWarriors() <<", "
00023     <<battle.get_TeutonicArmy().count_AliveWarriors() <<", "
00024     <<battle.get_Stats().get_HussarsDamage_Dealt() <<", "
00025     <<battle.get_Stats().get_TeutonicDamage_Dealt() <<", "
00026     <<battle.get_Stats().get_HussarsKills() <<", "
00027     <<battle.get_Stats().get_TeutonicKills() <<", "
00028     <<battle.get_Weather() <<", "
00029     <<"OK" <<", "
00030     <<date<<"\n";
00031     file.close();
00032 }
00033
00034 void BattleReport::save_skipped(const std::string& filename, int battleNumber, const std::string&
reason) {
00035     std::ofstream file(filename, std::ios::app);
00036
00037     file<<battleNumber<<", "
00038     <<"SKIPPED" <<", "
00039     <<"- " <<", "
00040     <<"- " <<", "
00041     <<"- " <<", "
00042     <<"- " <<", "
00043     <<"- " <<", "
00044     <<"- " <<", "
00045     <<"- " <<", "
00046     <<"- " <<", "
00047     <<reason<<", "
00048     <<"- " <<"\n";
00049 }

```

7.42 Random.cpp

```

00001 //
00002 // Created by bart on 28.05.2026.
00003 //
00004
00005
00006 #include <random>
00007 #include <utils/Random.hpp>
00008
00009 // Util for random number
00010
00011 int Random::random_Int(int min, int max) {
00012     if (min > max) {
00013         throw std::out_of_range("min must be less than max");
00014     }
00015
00016     static std::random_device rd;
00017     static std::mt19937 gen(rd());
00018     std::uniform_int_distribution<> numberRange(min, max);
00019
00020     return numberRange(gen);
00021 }
00022

```

7.43 Archer.cpp

```

00001 //
00002 // Created by bartl on 17.05.2026.
00003 //
00004
00005 #include <warriors/Archer.hpp>
00006 #include <core/UnitStats.hpp>
00007 #include "utils/Random.hpp"
00008
00009 // Archer(light unit) logic and attributes
00010
00011
00012 Archer::Archer(const UnitStats& stats) :
00013     Warrior(stats),
00014     atkBonus(1.25),
00015     defBonus(0.05)
00016 {}
00017
00018 double Archer::get_WeatherModifier(WeatherType weather) const {
00019     switch (weather) {
00020         case WeatherType::SUNNY:
00021             return 1.0;
00022         case WeatherType::RAINY:
00023             return 0.75;
00024         case WeatherType::FOGGY:
00025             return 0.65;
00026         case WeatherType::SNOWY:
00027             return 0.85;
00028     }
00029     return 1.0;
00030
00031 }
00032
00033 void Archer::attack_Enemy(Warrior& enemy, double moraleModifier, WeatherType weather) {
00034     double damage = (get_Attack()*moraleModifier*get_WeatherModifier(weather)*atkBonus);
00035     damage *= (100.0 / (100.0+enemy.get_Defense()));
00036
00037     enemy.take_Damage(damage);
00038
00039     if (Random::random_Int(1, 100) <= 20) {
00040         enemy.take_Damage(damage);
00041     }
00042
00043 }
00044
00045 void Archer::take_Damage(double damage) {
00046     damage *= (1.0-defBonus);
00047     Warrior::take_Damage(damage);
00048 };

```

7.44 Cavalryman.cpp

```

00001 //
00002 // Created by bartl on 17.05.2026.
00003 //
00004
00005 #include <warriors/Cavalryman.hpp>
00006 #include <core/UnitStats.hpp>
00007
00008 // Cavalryman(medium unit) logic and attributes
00009
00010
00011 Cavalryman::Cavalryman(const UnitStats& stats) :
00012     Warrior(stats),
00013     firstAttack(true),
00014     atkBonus(1.15),
00015     defBonus(0.10)
00016 {}
00017
00018 double Cavalryman::get_WeatherModifier(WeatherType weather) const {
00019     switch (weather) {
00020         case WeatherType::SUNNY:
00021             return 1.0;
00022         case WeatherType::RAINY:
00023             return 0.9;
00024         case WeatherType::FOGGY:
00025             return 0.85;
00026         case WeatherType::SNOWY:
00027             return 0.70;
00028     }
00029     return 1.0;

```

```

00030
00031 }
00032
00033 void Cavalryman::attack_Enemy(Warrior& enemy, double moraleModifier, WeatherType weather) {
00034     double damage = (get_Attack()*moraleModifier*get_WeatherModifier(weather)*atkBonus);
00035     damage *= (100.0 / (100.0+enemy.get_Defense()));
00036
00037     if (firstAttack) {
00038         damage*=2;
00039         firstAttack=false;
00040     }
00041
00042     enemy.take_Damage(damage);
00043 }
00044 }
00045
00046 void Cavalryman::take_Damage(double damage) {
00047     damage *= (1.0-defBonus);
00048     Warrior::take_Damage(damage);
00049 };

```

7.45 Knight.cpp

```

00001 //
00002 // Created by bartl on 17.05.2026.
00003 //
00004
00005 #include <warriors/Knight.hpp>
00006 #include <core/UnitStats.hpp>
00007
00008 // Knight (heavy unit) logic and attributes
00009
00010 Knight::Knight(const UnitStats& stats) :
00011     Warrior(stats),
00012     atkBonus(1.1),
00013     defBonus(0.20)
00014 {}
00015
00016 double Knight::get_WeatherModifier(WeatherType weather) const {
00017     switch (weather) {
00018         case WeatherType::SUNNY:
00019             return 1.0;
00020         case WeatherType::RAINY:
00021             return 0.95;
00022         case WeatherType::FOGGY:
00023             return 0.9;
00024         case WeatherType::SNOWY:
00025             return 0.85;
00026     }
00027     return 1.0;
00028 }
00029 }
00030
00031
00032 void Knight::attack_Enemy(Warrior& enemy, double moraleModifier, WeatherType weather) {
00033     double damage = (get_Attack()*moraleModifier*get_WeatherModifier(weather)*atkBonus);
00034     damage *= (100.0 / (100.0+enemy.get_Defense()));
00035
00036     enemy.take_Damage(damage);
00037 }
00038
00039
00040
00041 void Knight::take_Damage(double damage) {
00042     damage *= (1.0-defBonus);
00043     Warrior::take_Damage(damage);
00044 };

```

7.46 Warrior.cpp

```

00001 //
00002 // Created by bart on 14.05.2026.
00003 //
00004
00005 #include <warriors/Warrior.hpp>
00006 #include <core/UnitStats.hpp>
00007
00008

```

```
00009 // Abstract warrior class
00010
00011 Warrior::Warrior(UnitStats stats): health(stats.hp), attack(stats.attack), defense(stats.defense),
    alive(true)
00012 {};
00013
00014
00015 bool Warrior::is_Alive() const{
00016     return health>0;
00017 };
00018
00019 void Warrior::take_Damage(const double damage) {
00020     health -= damage;
00021     if (health <= 0) health = 0;
00022 };
00023
00024 // Getters
00025
00026 int Warrior::get_Health() const{
00027     return health;
00028 };
00029
00030 int Warrior::get_Attack() const{
00031     return attack;
00032 }
00033
00034 int Warrior::get_Defense() const {
00035     return defense;
00036 }
```


Index

- add_HussarsDamage
 - BattleStats, [35](#)
- add_HussarsKill
 - BattleStats, [35](#)
- add_Reinforcements
 - Army, [18](#)
- add_TeutonicDamage
 - BattleStats, [35](#)
- add_TeutonicKill
 - BattleStats, [35](#)
- addBattleResult
 - HistoryPanel, [51](#)
- AlliedHelp, [11](#)
 - apply, [12](#)
 - getName, [12](#)
- apply
 - AlliedHelp, [12](#)
 - CommanderDeath, [42](#)
 - PanicEvent, [62](#)
 - RandomEvent, [65](#)
- Archer, [13](#)
 - Archer, [15](#)
 - attack_Enemy, [15](#)
 - get_Type, [15](#)
 - get_WeatherModifier, [16](#)
 - take_Damage, [16](#)
- archer_count
 - FractionConfig, [48](#)
- archer_stats
 - FractionConfig, [48](#)
- Army, [17](#)
 - add_Reinforcements, [18](#)
 - Army, [18](#)
 - count_AliveArchers, [18](#)
 - count_AliveCavalry, [19](#)
 - count_AliveKnights, [19](#)
 - count_AliveWarriors, [19](#)
 - decrease_Morale, [19](#)
 - get_ArmySize, [19](#)
 - get_moraleLevel, [19](#)
 - get_MoraleModifier, [20](#)
 - get_Name, [20](#)
 - get_RandomAliveWarrior, [20](#)
 - get_Warriors, [20](#)
 - increase_Morale, [20](#)
- attach
 - Battle, [23](#)
- attack
 - UnitStats, [70](#)
- attack_Enemy
 - Archer, [15](#)
 - Cavalryman, [39](#)
 - Knight, [58](#)
 - Warrior, [73](#)
- Battle, [21](#)
 - attach, [23](#)
 - Battle, [22](#)
 - do_Round, [23](#)
 - finishBattle, [23](#)
 - get_CurrentEvent, [23](#)
 - get_HussarsArmy, [23](#)
 - get_Rounds, [23](#)
 - get_Stats, [24](#)
 - get_TeutonicArmy, [24](#)
 - get_Weather, [24](#)
 - get_Winner, [24](#)
 - is_finished, [24](#)
 - notify, [24](#)
 - start_Battle, [25](#)
- BattleObserver, [25](#)
 - update, [27](#)
- BattlePanel, [27](#)
 - BattlePanel, [30](#)
 - setBattleNumber, [30](#)
 - setCurrentEvent, [30](#)
 - setHussarsMorale, [30](#)
 - setHussarsProbability, [31](#)
 - setRounds, [31](#)
 - setTeutonicMorale, [31](#)
 - setTeutonicProbability, [31](#)
 - setWeather, [31](#)
 - setWinner, [32](#)
 - update, [32](#)
- BattleReport, [32](#)
 - save, [33](#)
 - save_skipped, [33](#)
- BattleStats, [34](#)
 - add_HussarsDamage, [35](#)
 - add_HussarsKill, [35](#)
 - add_TeutonicDamage, [35](#)
 - add_TeutonicKill, [35](#)
 - get_HussarsDamage_Dealt, [35](#)
 - get_HussarsKills, [35](#)
 - get_TeutonicDamage_Dealt, [36](#)
 - get_TeutonicKills, [36](#)
- cavalry_count
 - FractionConfig, [48](#)

- cavalry_stats
 - FractionConfig, 48
- Cavalryman, 36
 - attack_Enemy, 39
 - Cavalryman, 39
 - get_Type, 39
 - get_WeatherModifier, 40
 - take_Damage, 40
- clearHistoryList
 - HistoryPanel, 51
- CommanderDeath, 41
 - apply, 42
 - getName, 42
- ConfigLoader, 43
 - LoadFromFile, 43
- ControlPanel, 44
 - ControlPanel, 45
 - get_ExitButton, 46
 - get_LoadButton, 46
 - get_SpeedComboBox, 46
 - get_StartButton, 46
- count_AliveArchers
 - Army, 18
- count_AliveCavalry
 - Army, 19
- count_AliveKnights
 - Army, 19
- count_AliveWarriors
 - Army, 19
- decrease_Morale
 - Army, 19
- defense
 - UnitStats, 70
- do_Round
 - Battle, 23
- finishBattle
 - Battle, 23
- fraction
 - FractionConfig, 49
- FractionConfig, 47
 - archer_count, 48
 - archer_stats, 48
 - cavalry_count, 48
 - cavalry_stats, 48
 - fraction, 49
 - knight_count, 49
 - knight_stats, 49
 - valid, 49
- get_ArmySize
 - Army, 19
- get_Attack
 - Warrior, 73
- get_CurrentEvent
 - Battle, 23
- get_Defense
 - Warrior, 73
- get_ExitButton
 - ControlPanel, 46
- get_Health
 - Warrior, 74
- get_HussarsArmy
 - Battle, 23
- get_HussarsDamage_Dealt
 - BattleStats, 35
- get_HussarsKills
 - BattleStats, 35
- get_LoadButton
 - ControlPanel, 46
- get_moraleLevel
 - Army, 19
- get_MoraleModifier
 - Army, 20
- get_Name
 - Army, 20
- get_RandomAliveWarrior
 - Army, 20
- get_Rounds
 - Battle, 23
- get_SpeedComboBox
 - ControlPanel, 46
- get_StartButton
 - ControlPanel, 46
- get_Stats
 - Battle, 24
- get_TeutonicArmy
 - Battle, 24
- get_TeutonicDamage_Dealt
 - BattleStats, 36
- get_TeutonicKills
 - BattleStats, 36
- get_Type
 - Archer, 15
 - Cavalryman, 39
 - Knight, 58
 - Warrior, 74
- get_Warriors
 - Army, 20
- get_Weather
 - Battle, 24
- get_WeatherModifier
 - Archer, 16
 - Cavalryman, 40
 - Knight, 59
 - Warrior, 74
- get_WeatherType
 - Weather, 76
- get_Winner
 - Battle, 24
- getName
 - AlliedHelp, 12
 - CommanderDeath, 42
 - PanicEvent, 62
 - RandomEvent, 65
- HistoryPanel, 50

- addBattleResult, 51
- clearHistoryList, 51
- HistoryPanel, 51
- hp
 - UnitStats, 70
- HussarsPanel, 52
 - HussarsPanel, 54
 - setAlive, 54
 - setArchers, 54
 - setCavalrymans, 54
 - setDamage, 54
 - setKills, 54
 - setKnights, 55
- include/config/ConfigLoader.hpp, 77
- include/config/FractionConfig.hpp, 77
- include/core/Army.hpp, 78
- include/core/Battle.hpp, 78
- include/core/BattleStats.hpp, 79
- include/core/UnitStats.hpp, 80
- include/events/AlliedHelp.hpp, 80
- include/events/CommanderDeath.hpp, 80
- include/events/PanicEvent.hpp, 80
- include/events/RandomEvent.hpp, 81
- include/events/Weather.hpp, 81
- include/events/WeatherType.hpp, 81
- include/gui/BattlePanel.hpp, 81
- include/gui/ControlPanel.hpp, 82
- include/gui/HistoryPanel.hpp, 83
- include/gui/HussarsPanel.hpp, 83
- include/gui/MainWindow.hpp, 83
- include/gui/TeutonicPanel.hpp, 84
- include/observer/BattleObserver.hpp, 85
- include/reports/BattleReport.hpp, 85
- include/utils/Random.hpp, 85
- include/warriors/Archer.hpp, 86
- include/warriors/Cavalryman.hpp, 86
- include/warriors/Knight.hpp, 86
- include/warriors/Warrior.hpp, 87
- increase_Morale
 - Army, 20
- is_Alive
 - Warrior, 74
- is_finished
 - Battle, 24
- Knight, 55
 - attack_Energy, 58
 - get_Type, 58
 - get_WeatherModifier, 59
 - Knight, 58
 - take_Damage, 59
- knight_count
 - FractionConfig, 49
- knight_stats
 - FractionConfig, 49
- LoadFromFile
 - ConfigLoader, 43
- MainWindow, 60
 - MainWindow, 61
- notify
 - Battle, 24
- PanicEvent, 61
 - apply, 62
 - getName, 62
- Random, 63
 - random_Int, 64
- random_Int
 - Random, 64
- RandomEvent, 64
 - apply, 65
 - getName, 65
- save
 - BattleReport, 33
- save_skipped
 - BattleReport, 33
- setAlive
 - HussarsPanel, 54
 - TeutonicPanel, 68
- setArchers
 - HussarsPanel, 54
 - TeutonicPanel, 68
- setBattleNumber
 - BattlePanel, 30
- setCavalrymans
 - HussarsPanel, 54
 - TeutonicPanel, 68
- setCurrentEvent
 - BattlePanel, 30
- setDamage
 - HussarsPanel, 54
 - TeutonicPanel, 68
- setHussarsMorale
 - BattlePanel, 30
- setHussarsProbability
 - BattlePanel, 31
- setKills
 - HussarsPanel, 54
 - TeutonicPanel, 68
- setKnights
 - HussarsPanel, 55
 - TeutonicPanel, 69
- setRounds
 - BattlePanel, 31
- setTeutonicMorale
 - BattlePanel, 31
- setTeutonicProbability
 - BattlePanel, 31
- setWeather
 - BattlePanel, 31
- setWinner
 - BattlePanel, 32
- Simulation Battle of Grunwald, 1, 3

- src/config/ConfigLoader.cpp, 88
- src/core/Army.cpp, 91
- src/core/Battle.cpp, 92
- src/core/BattleStats.cpp, 95
- src/events/AlliedHelp.cpp, 95
- src/events/CommanderDeath.cpp, 95
- src/events/PanicEvent.cpp, 96
- src/events/Weather.cpp, 96
- src/gui/BattlePanel.cpp, 97
- src/gui/ControlPanel.cpp, 98
- src/gui/HistoryPanel.cpp, 99
- src/gui/HussarsPanel.cpp, 99
- src/gui/MainWindow.cpp, 101
- src/gui/TeutonicPanel.cpp, 105
- src/reports/BattleReport.cpp, 107
- src/utils/Random.cpp, 107
- src/warriors/Archer.cpp, 108
- src/warriors/Cavalryman.cpp, 108
- src/warriors/Knight.cpp, 109
- src/warriors/Warrior.cpp, 109
- start_Battle
 - Battle, 25
- take_Damage
 - Archer, 16
 - Cavalryman, 40
 - Knight, 59
 - Warrior, 74
- TeutonicPanel, 66
 - setAlive, 68
 - setArchers, 68
 - setCavalrymans, 68
 - setDamage, 68
 - setKills, 68
 - setKnights, 69
 - TeutonicPanel, 68
- to_String
 - Weather, 76
- UnitStats, 69
 - attack, 70
 - defense, 70
 - hp, 70
- update
 - BattleObserver, 27
 - BattlePanel, 32
- valid
 - FractionConfig, 49
- Warrior, 71
 - attack_Enemy, 73
 - get_Attack, 73
 - get_Defense, 73
 - get_Health, 74
 - get_Type, 74
 - get_WeatherModifier, 74
 - is_Alive, 74
 - take_Damage, 74
 - Warrior, 73
- Weather, 75
 - get_WeatherType, 76
 - to_String, 76
 - Weather, 76